ReactJS Interview Guide

JavaScript, Hooks, Patterns, State, Performance, Machine Coding

Theory + practical + real-world scenario questions, with solutions

Interview Prep Series | Prepared for Mustafa

Contents

1. JavaScript Foundations Interviewers Test Before React 3
2. React Core Concepts 5
3. Hooks in Depth 7
4. Component Patterns and Architecture 11
5. State Management 13
6. Performance Optimisation 15
7. Routing, Forms, API and Security 16
8. Rendering Strategies, Testing and Tooling 18
9. Output-Based and Tricky Questions 19
10. Machine-Coding / Practical Rounds 20
11. Real-World Scenario Questions 23
12. Quick-Fire Questions (Short Answers) 25
13. JavaScript Deep Dive and Tricky Output Questions 27
14. React Core: More Concepts and Tricky Questions 33
15. Hooks: Tricky and Advanced Questions 37
16. Rendering, Reconciliation and Performance (Advanced) 40
17. State Management (Advanced) 43
18. Routing, Forms and API Integration (More) 46
19. TypeScript with React 49
20. Testing React (More) 51
21. Next.js and Server-Side React 53
22. Security, Accessibility and Browser Questions 55
23. More Output-Based React Puzzles 57
24. More Machine-Coding Problems 60
25. Real-World Scenarios (More) 63
26. Experience and Project Discussion Questions 66
27. Rapid-Fire Round (One-Liners) 67
28. Bonus Tricky Questions 70

1. JavaScript Foundations Interviewers Test Before React

Q1. What is the difference between var, let and const?

`var` is function-scoped, hoisted and initialised to `undefined`, and can be redeclared. `let` and `const` are block-scoped and hoisted into the Temporal Dead Zone (TDZ), so touching them before the declaration throws a ReferenceError. `const` prevents re-assignment of the binding, not mutation of the object it points to.

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2 (new binding per iteration)
const user = { name: "A" }; user.name = "B"; // allowed
```

> **Interview tip:** In React, this is the root cause of many "stale value" bugs in loops and callbacks. Mention per-iteration bindings of `let`.

Q2. Explain closures with a practical example.

A closure is a function that remembers variables from the scope in which it was created, even after that scope has finished executing. React hooks depend on closures: each render creates new functions that close over that render's props and state.

```
function makeCounter() {
  let count = 0;                    // private
  return { inc: () => ++count, get: () => count };
}
const c = makeCounter(); c.inc(); c.get(); // 1
```

Real-world: debounce, memoisation, module privacy, and event handlers that read state.

Q3. What is the event loop? Order the output.

JS is single-threaded. Synchronous code runs on the call stack; when it empties, the engine drains the **microtask queue** (Promises, queueMicrotask, MutationObserver) completely, then takes one **macrotask** (setTimeout, I/O, UI events), then microtasks again, then may render.

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
queueMicrotask(() => console.log(4));
console.log(5);
// Output: 1 5 3 4 2
```

Q4. Explain `this` in JS and how arrow functions differ.

`this` is decided by how a function is called: method call (object), plain call (`undefined` in strict mode), `new` (new object), `call/apply/bind` (explicit). Arrow functions have no own `this`; they capture it lexically. This is why class components needed `this.handleClick = this.handleClick.bind(this)` or arrow class fields.

Q5. Shallow copy vs deep copy - why does it matter in React?

React compares state by reference (`Object.is`). Mutating an object and calling `setState(sameObj)` does nothing because the reference did not change. Spread (`{...obj}`) is a shallow copy; nested objects are still shared. Use `structuredClone` for deep copy, or update immutably at each level.

```
setUser(prev => ({ ...prev, address: { ...prev.address, city: "Pune" } }));
```

Q6. What are debounce and throttle? Implement debounce.

Debounce: run only after the user stops triggering for N ms (search box). Throttle: run at most once every N ms (scroll, resize).

```
function debounce(fn, delay) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), delay);
  };
}
function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
}
```

Q7. Promise.all vs allSettled vs race vs any.

- `all`: resolves when all resolve; rejects on the first rejection (fail-fast).
- `allSettled`: waits for all; gives status of each. Good for dashboards where one widget may fail.
- `race`: settles with the first settled promise (useful for timeouts).
- `any`: resolves with the first fulfilled; rejects only if all reject.

Q8. What is the difference between == and ===, and null vs undefined?

`===` compares without type coercion; `==` coerces (`"0" == 0` is true). `undefined` means not assigned; `null` is an intentional empty value. `null == undefined` is true, `null === undefined` is false. Use optional chaining `a?.b` and nullish coalescing `x ?? "default"` (only falls back for null/undefined, unlike `||` which also replaces 0 and "").

2. React Core Concepts

Q9. What is React and why use it?

React is a JavaScript library for building UIs from components. Key ideas: declarative UI (describe what the screen should look like for a given state), component composition, one-way data flow, and an efficient reconciliation process that updates only what changed. It is a library, not a full framework; routing, data fetching and forms come from the ecosystem (React Router, TanStack Query, React Hook Form) or a meta-framework like Next.js.

Q10. What is the Virtual DOM and how does reconciliation work?

The Virtual DOM is a lightweight JS tree describing the UI. On each render React produces a new tree and diffs it against the previous one (reconciliation), then applies the minimal set of real DOM operations (commit). The diffing heuristics are O(n):

- Different element types: tear down the old subtree and build a new one.
- Same type: keep the DOM node and update changed attributes.
- Lists: match children by `key`.

> **Interview tip:** Modern answer: since React 16 the reconciler is **Fiber**, which splits rendering into units of work so React can pause, prioritise and resume (basis of concurrent features).

Q11. What is React Fiber?

Fiber is the reimplementation of React's reconciler. Each component instance is a fiber node with links (child, sibling, return). Work is split into a **render phase** (pure, interruptible, can be thrown away) and a **commit phase** (synchronous, mutates the DOM, runs layout effects). This enables time-slicing, `startTransition`, Suspense and priority-based updates.

Q12. What is JSX? Can you use React without it?

JSX is syntax sugar compiled to function calls. `<Button type="x">Hi</Button>` becomes `jsx(Button, { type: "x", children: "Hi" })` (new JSX transform) or `React.createElement(...)` in older setups. You can write React without JSX but it is verbose. JSX expressions must return a single root, so use a Fragment `<>...</>` when needed.

Q13. Functional vs class components?

Function components plus hooks are the modern standard: less boilerplate, easier logic reuse through custom hooks, no `this` confusion. Class components use lifecycle methods (`componentDidMount`, etc.). The only thing still requiring a class is an **Error Boundary** (unless you use a library like `react-error-boundary`).

Q14. Props vs state.

Props are inputs passed from the parent; read-only inside the child. State is data owned by the component that changes over time and triggers re-render. If a value can be computed from props or other state, it should be neither: compute it during render (derived data).

Q15. Why are keys important in lists? Why is index a bad key?

Keys let React match old and new children. If you use the array index and the list is reordered, filtered or has items inserted at the top, React reuses the wrong component instances, so local state (input text, checkbox) sticks to the wrong row.

```
{orders.map(o => <OrderRow key={o.orderId} order={o} />)}  // stable unique id
```

Index is acceptable only for static lists that never reorder.

> **Interview tip:** Trick: changing a component's key forces React to remount it, which is a clean way to reset its state, e.g. `<Profile key={userId} />`.

Q16. Controlled vs uncontrolled components.

Controlled: the input value lives in React state and changes via `onChange`; React is the single source of truth (validation on every keystroke, conditional fields). Uncontrolled: the DOM keeps the value; you read it with a ref (`useRef`) or FormData on submit. Uncontrolled is lighter for big forms; React Hook Form uses this approach for performance.

Q17. What is lifting state up?

When two siblings need the same data, move the state to their closest common parent and pass it down as props plus callbacks. Example: a filter panel and a results grid on a reconciliation screen both read `filters` stored in the page component.

Q18. What is prop drilling and how do you avoid it?

Passing props through many intermediate components that do not use them. Solutions: component composition (pass `children` or elements as props), Context API for low-frequency global data (theme, auth user, locale), or a state library (Redux Toolkit, Zustand) for frequently changing shared state.

Q19. What are Fragments and why use them?

`<></>` or `<React.Fragment key=...>` group children without adding an extra DOM node. This keeps valid HTML (e.g. multiple `<td>` inside a `<tr>`) and avoids extra wrappers that break flex/grid layouts. Only the long form accepts a `key`.

Q20. What is conditional rendering? Any gotchas?

Use `if`, ternary, or `&&`. Gotcha: `{count && <Badge/>}` renders a `0` when count is 0. Use `{count > 0 && <Badge/>}`.

Q21. What does "one-way data flow" mean?

Data flows from parent to child via props. Children communicate upward by calling callback props. This makes state changes predictable and easier to debug than two-way binding.

Q22. What happens when you call setState? Is it synchronous?

It schedules an update; the state variable in the current render does not change. React re-renders later and the next render reads the new value. Since React 18, all updates (including in timeouts, promises, native events) are **automatically batched**.

```
const [n, setN] = useState(0);
function add3() {
  setN(n + 1); setN(n + 1); setN(n + 1);  // result: 1 (all read the same n)
  setN(p => p + 1); setN(p => p + 1);      // functional form: each sees the latest
}
```

Q23. What causes a component to re-render?

1) Its state changes, 2) its parent re-renders (even if props are the same), 3) a context it consumes changes. Props changing is not itself a trigger; the parent re-rendering is. To skip, wrap with `React.memo` and keep props referentially stable.

3. Hooks in Depth

Q24. Rules of Hooks and why they exist.

Call hooks only at the top level (not inside loops, conditions or nested functions) and only from function components or custom hooks. React identifies each hook by **call order** in a linked list on the fiber; a conditional hook shifts the order and assigns the wrong state to the wrong hook.

Q25. useState: lazy initialisation.

If the initial value is expensive, pass a function so it runs only on the first render.

```
const [rows, setRows] = useState(() => JSON.parse(localStorage.getItem("rows") ?? "[]"));
```

Q26. Explain useEffect and its dependency array.

`useEffect(fn, deps)` runs after the browser paints. No array: after every render. `[]`: once after mount (twice in dev with StrictMode). `[a, b]`: when `a` or `b` change (compared with `Object.is`). Return a cleanup function, which runs before the next effect and on unmount.

```
useEffect(() => {
  const id = setInterval(poll, 5000);
  return () => clearInterval(id);       // cleanup prevents leaks
}, [poll]);
```

> **Interview tip:** Say this line: "Effects are for synchronising with external systems. If there is no external system, you probably don't need an effect."

Q27. Why does useEffect run twice in development?

React 18 StrictMode mounts, unmounts and remounts components in development to surface missing cleanups. It does not happen in production. Fix the effect (add cleanup / abort fetch) rather than removing StrictMode.

Q28. How do you fetch data in useEffect correctly and avoid race conditions?

```
useEffect(() => {
  const ctrl = new AbortController();
  setLoading(true);
  fetch(`/api/txn?date=${date}`, { signal: ctrl.signal })
    .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
    .then(setData)
    .catch(e => { if (e.name !== "AbortError") setError(e); })
    .finally(() => setLoading(false));
  return () => ctrl.abort();   // old request cancelled when date changes
}, [date]);
```

In real projects prefer TanStack Query / RTK Query: caching, dedupe, retries, background refetch.

Q29. useEffect vs useLayoutEffect.

`useLayoutEffect` runs synchronously after DOM mutation but **before paint**. Use it to measure the DOM (tooltip position, scroll restore) to avoid visible flicker. It blocks painting, so default to `useEffect`.

Q30. What is useRef used for?

1) Access DOM nodes (`inputRef.current.focus()`). 2) Store mutable values that persist across renders without causing re-render: timer IDs, previous value, latest callback, "is mounted" flags.

```
function usePrevious(v) {
  const ref = useRef();
  useEffect(() => { ref.current = v; });
  return ref.current;
}
```

Q31. useMemo vs useCallback.

`useMemo(() => compute(a), [a])` caches a **value**. `useCallback(fn, [a])` caches a **function reference** (same as `useMemo(() => fn, [a])`). They help only when: the computation is genuinely expensive, or a stable reference is needed for a `React.memo` child or as an effect dependency. Overusing them adds cost and noise.

> **Interview tip:** React Compiler (React 19 era) auto-memoises, reducing the need for manual useMemo/useCallback. Mention you know it exists.

Q32. What is useReducer and when do you prefer it over useState?

When the next state depends on complex logic or multiple sub-values change together (form wizards, cart, filter panels). Reducers are pure and testable.

```
function reducer(state, action) {
  switch (action.type) {
    case "add": return { ...state, items: [...state.items, action.item] };
    case "remove": return { ...state, items: state.items.filter(i => i.id !== action.id) };
    case "reset": return initial;
    default: throw new Error("Unknown action " + action.type);
  }
}
const [state, dispatch] = useReducer(reducer, initial);
```

Q33. Explain useContext and its performance pitfall.

Context avoids prop drilling. Every consumer re-renders when the Provider's `value` changes by reference. Pitfall: `value={{ user, setUser }}` creates a new object each render, re-rendering all consumers. Fixes: memoise value, split contexts (state vs dispatch), or use a store with selectors (Zustand/Redux) for high-frequency data.

```
const value = useMemo(() => ({ user, login, logout }), [user]);
return <AuthCtx.Provider value={value}>{children}</AuthCtx.Provider>;
```

Q34. What are custom hooks? Write useFetch and useDebounce.

A custom hook is a function starting with `use` that calls other hooks, to reuse stateful logic (not state itself; each caller gets its own state).

```
function useDebounce(value, delay = 400) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setV(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return v;
}

function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  useEffect(() => {
    const c = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: c.signal })
      .then(r => r.json())
      .then(data => setState({ data, loading: false, error: null }))
      .catch(error => error.name !== "AbortError" && setState({ data: null, loading: false, error }));
    return () => c.abort();
  }, [url]);
  return state;
}
```

Q35. What is a stale closure and how do you fix it?

A callback captured an old render's state. Classic example: interval reading `count` with an empty dependency array always sees 0.

```
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000); // functional update fixes it
  return () => clearInterval(id);
}, []);
```

Other fixes: include the value in deps, or keep the latest value in a ref.

Q36. What are useTransition and useDeferredValue?

Concurrent features (React 18). `useTransition` marks a state update as non-urgent so typing stays responsive while a heavy list re-renders. `useDeferredValue` gives a lagging copy of a value to render the expensive part with.

```
const [isPending, startTransition] = useTransition();
function onChange(e) {
  setQuery(e.target.value);                          // urgent
  startTransition(() => setFilter(e.target.value)); // can be interrupted
}
```

Q37. What are useId, useImperativeHandle and forwardRef?

- `useId`: stable unique ids for accessibility attributes that match between server and client.
- `forwardRef`: lets a parent pass a ref through to a child's DOM node (in React 19, `ref` is a normal prop for function components).
- `useImperativeHandle`: exposes a limited custom API instead of the raw DOM node, e.g. `{ focus, clear }`.

Q38. What is useSyncExternalStore?

A hook to subscribe to external stores (browser APIs, Redux, Zustand) safely under concurrent rendering, avoiding "tearing" where different parts of the UI show different versions of the store.

Q39. What are the new hooks in React 19?

`use()` (read a promise or context, can be called conditionally), `useActionState` (form action state + pending), `useFormStatus`, `useOptimistic` (show optimistic UI until the server confirms). Also Actions: async functions passed to `<form action={fn}>`, plus ref as a prop and document metadata support.

4. Component Patterns and Architecture

Q40. What is a Higher-Order Component (HOC)?

A function that takes a component and returns an enhanced component. Used for cross-cutting concerns (auth guard, logging). Mostly replaced by hooks today.

```
const withAuth = (Comp) => (props) => {
  const { user } = useAuth();
  return user ? <Comp {...props} /> : <Navigate to="/login" />;
};
```

Q41. What is the render props pattern?

A component receives a function as a prop and calls it to decide what to render. Used by older libraries (Formik, React Motion). Hooks usually express the same thing more simply.

Q42. Compound components pattern?

Components that share implicit state via context, giving a flexible API: `<Tabs><Tabs.List/><Tabs.Panel/></Tabs>`. Used in design systems (Radix, Headless UI).

Q43. Container vs presentational components?

Container handles data fetching and state; presentational only renders props. Hooks blurred this, but the principle of separating data logic (custom hooks) from UI remains good practice.

Q44. How would you structure a large React project?

Feature-based folders instead of type-based:

```
src/
  app/            (providers, router, store)
  features/
    reconciliation/ (components, hooks, api, slice, types, tests)
    settlement/
  shared/          (ui kit, utils, hooks, api client)
  pages/
```

Keep API calls in one client layer (axios instance with interceptors), use absolute imports, TypeScript, ESLint + Prettier, and colocate tests.

Q45. What are Error Boundaries? What do they not catch?

Class components implementing `static getDerivedStateFromError` and/or `componentDidCatch` that catch errors during rendering, lifecycle and constructors of children, and show a fallback UI. They do **not** catch: event handler errors, async code (setTimeout, promises), SSR errors, or errors in the boundary itself.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(err, info) { logToServer(err, info.componentStack); }
  render() { return this.state.hasError ? <Fallback /> : this.props.children; }
}
```

Q46. What are Portals?

`createPortal(child, domNode)` renders children into a different DOM node (modals, tooltips) to escape `overflow:hidden`/z-index stacking, while events still bubble through the React tree.

Q47. What is Suspense?

`<Suspense fallback={<Spinner/>}>` shows a fallback while a child is "suspended", waiting for lazy-loaded code (`React.lazy`) or data from a Suspense-enabled source (Next.js, TanStack Query suspense mode, React 19 `use`).

5. State Management

Q48. When do you need Redux? What are alternatives?

Use a global store when many distant components read and update the same frequently-changing data, you need devtools/time-travel, or a strict update pattern for a big team. Separate **server state** (API data: use TanStack Query/RTK Query) from **client state** (UI: local state, Context, Zustand). Many apps need no Redux once server state is handled by a query library.

Q49. Explain the Redux data flow.

UI dispatches an **action** -> store runs the **reducer**(state, action) -> new state -> subscribed components (via `useSelector`) re-render if their selected slice changed. Reducers must be pure and immutable. Side effects go in thunks, sagas or RTK Query.

Q50. Show a Redux Toolkit slice with an async thunk.

```
export const fetchTxns = createAsyncThunk("txn/fetch", async (date, { rejectWithValue }) => {
  try { return (await api.get(`/txn?date=${date}`)).data; }
  catch (e) { return rejectWithValue(e.response?.data); }
});

const txnSlice = createSlice({
  name: "txn",
  initialState: { items: [], status: "idle", error: null },
  reducers: {
    clear: (s) => { s.items = []; }  // Immer lets us "mutate" safely
  },
  extraReducers: (b) => {
    b.addCase(fetchTxns.pending, s => { s.status = "loading"; })
     .addCase(fetchTxns.fulfilled, (s, a) => { s.status = "ok"; s.items = a.payload; })
     .addCase(fetchTxns.rejected, (s, a) => { s.status = "failed"; s.error = a.payload; });
  }
});
```

Q51. What is Immer and why can RTK reducers "mutate"?

RTK uses Immer, which wraps state in a Proxy, records your mutations to a draft and produces a new immutable object. So `s.items.push(x)` is safe inside `createSlice` but nowhere else.

Q52. What are selectors and reselect?

Selectors extract data from the store. `createSelector` memoises derived data so `useSelector` returns the same reference when inputs didn't change, avoiding unnecessary re-renders.

```
const selectMatched = createSelector([s => s.txn.items], items => items.filter(i => i.status ===
    "MATCHED"));
```

Q53. Context vs Redux vs Zustand.

Context: dependency injection, not a state manager; all consumers re-render on change. Redux Toolkit: structured, middleware, devtools, best for large teams. Zustand: tiny store with selector-based subscriptions, minimal boilerplate.

Q54. How does TanStack (React) Query help?

It manages server state: caching by query key, background refetching, stale-while-revalidate, retries, pagination, optimistic updates and invalidation after mutations.

```
const { data, isLoading } = useQuery({ queryKey: ["txn", date], queryFn: () => getTxns(date), staleTime:
    60000 });
const mut = useMutation({ mutationFn: approve, onSuccess: () => qc.invalidateQueries({ queryKey: ["txn"]
    }) });
```

6. Performance Optimisation

Q55. How do you find performance problems in React?

React DevTools Profiler (which components rendered, why, how long), "Highlight updates", Chrome Performance tab, Lighthouse / Web Vitals (LCP, INP, CLS), and `why-did-you-render` in dev. Measure first, then optimise.

Q56. What is React.memo? When does it not help?

`React.memo(Comp)` skips re-rendering when props are shallow-equal. It fails if you pass new object/array/function references every render, or children elements. Pair it with useMemo/useCallback for those props.

```
const Row = React.memo(function Row({ item, onSelect }) { ... });
const onSelect = useCallback(id => setSelected(id), []);
```

Q57. How do you render 100,000 rows efficiently?

Virtualisation (windowing): render only visible rows with `react-window`, `react-virtuoso` or TanStack Virtual. Also server-side pagination/filtering, memoised rows, stable keys, avoid heavy inline computations, and use Web Workers for heavy parsing (e.g. a large CSV before reconciliation).

Q58. What is code splitting and lazy loading?

Split the bundle so users download only what they need.

```
const Reports = React.lazy(() => import("./pages/Reports"));
<Suspense fallback={<Loader/>}>
  <Routes><Route path="/reports" element={<Reports/>} /></Routes>
</Suspense>
```

Route-based splitting is the biggest win. Also: tree shaking, analysing bundles (`vite-bundle-visualizer`), compressing images, and lazy loading images.

Q59. List common performance techniques.

- Keep state as local as possible (colocate); a state change at the top re-renders everything below.
- Move expensive parts into children so they're not re-created (composition, passing `children`).
- Memoise expensive computations and stable callbacks for memoised children.
- Virtualise long lists; paginate on the server.
- Debounce search input; use `useTransition` for heavy updates.
- Avoid creating context values inline; split contexts.
- Lazy load routes and heavy libraries (charts, editors).

7. Routing, Forms, API and Security

Q60. How does React Router v6 work? Protected routes?

```
<BrowserRouter>
  <Routes>
    <Route path="/login" element={<Login/>} />
    <Route element={<RequireAuth/>}>
      <Route path="/" element={<Layout/>}>
        <Route index element={<Dashboard/>} />
        <Route path="txn/:id" element={<TxnDetail/>} />
      </Route>
    </Route>
    <Route path="*" element={<NotFound/>} />
  </Routes>
</BrowserRouter>

function RequireAuth() {
  const { user } = useAuth(); const loc = useLocation();
  return user ? <Outlet/> : <Navigate to="/login" state={{ from: loc }} replace />;
}
```

Hooks: `useParams`, `useNavigate`, `useSearchParams`, `useLocation`. Remember: frontend guards are UX only; the API must authorise every request.

Q61. How do you handle forms and validation at scale?

React Hook Form (uncontrolled inputs, minimal re-renders) with a schema validator like Zod or Yup. Show field errors, disable submit while pending, and validate again on the server.

```
const schema = z.object({ amount: z.number().positive(), rrn: z.string().length(12) });
const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({ resolver: zodResolver(
    schema) });
```

Q62. How do you set up an API layer with JWT and refresh tokens?

A single axios instance: request interceptor attaches the access token; response interceptor catches 401, calls refresh once (queueing concurrent requests), retries, or logs out.

```
api.interceptors.response.use(r => r, async (err) => {
  const orig = err.config;
  if (err.response?.status === 401 && !orig._retry) {
    orig._retry = true;
    refreshPromise ??= api.post("/auth/refresh").finally(() => (refreshPromise = null));
    await refreshPromise;
    return api(orig);
  }
  return Promise.reject(err);
});
```

Q63. Where should you store JWTs?

Safest common pattern: short-lived access token in memory, refresh token in an **HttpOnly, Secure, SameSite** cookie (JS cannot read it, so XSS can't steal it). localStorage is readable by any injected script. With cookies, protect against CSRF (SameSite, CSRF token).

Q64. How does React protect against XSS? When are you still vulnerable?

JSX escapes values by default. You are vulnerable with `dangerouslySetInnerHTML` (sanitise with DOMPurify), `href={userInput}` allowing `javascript:` URLs, and third-party scripts. Add a Content Security Policy.

Q65. How do you call a .NET Core API from React and handle CORS?

CORS is enforced by the browser; the server must allow the origin. In ASP.NET Core: `builder.Services.AddCors(o => o.AddPolicy("ui", p => p.WithOrigins("https://app.example.com").AllowAnyHeader().AllowAnyMethod().AllowCredentials()));` then `app.UseCors("ui")`. In dev, use the Vite/CRA proxy to avoid CORS entirely.

8. Rendering Strategies, Testing and Tooling

Q66. CSR vs SSR vs SSG vs ISR vs RSC.

- CSR: browser downloads JS and renders; slower first paint, bad for SEO.
- SSR: server renders HTML per request, then hydration attaches events.
- SSG: HTML built at build time; fastest, for static content.
- ISR (Next.js): static pages regenerated in the background after a time.
- React Server Components: components run only on the server, send no JS for themselves, can access DB directly; client components marked with `"use client"`.

Q67. What is hydration and what causes hydration mismatch?

Hydration attaches React to server-rendered HTML. Mismatch happens when the client's first render differs from the server HTML: `Date.now()`, `Math.random()`, `window` checks, locale formatting. Fix by moving client-only values into `useEffect` or using `useId`.

Q68. How do you test React components?

Jest or Vitest + React Testing Library: test behaviour like a user (find by role/label, click, assert text), not implementation details. Mock network with MSW. E2E with Playwright/Cypress.

```
test("shows error on invalid amount", async () => {
  render(<PaymentForm />);
  await userEvent.type(screen.getByLabelText(/amount/i), "-5");
  await userEvent.click(screen.getByRole("button", { name: /submit/i }));
  expect(await screen.findByText(/must be positive/i)).toBeInTheDocument();
});
```

Q69. Why use TypeScript with React? Show typed props.

Catches bugs at compile time, documents component APIs, improves refactoring.

```
type Props = { txn: Txn; onApprove: (id: string) => void; variant?: "compact" | "full" };
export function TxnCard({ txn, onApprove, variant = "full" }: Props) { ... }
function useList<T>(items: T[]) { ... }   // generics in hooks
```

Q70. Vite vs Create React App?

CRA is deprecated. Vite uses native ES modules in dev (near-instant startup, fast HMR) and Rollup for production builds. Environment variables are `import.meta.env.VITE_*`.

9. Output-Based and Tricky Questions

Q71. What renders after clicking once?

```
function App() {
  const [c, setC] = useState(0);
  return <button onClick={() => { setC(c + 5); setC(c => c + 1); }}>{c}</button>;
}
```

6. The first update queues "replace with 5"; the functional update then runs on 5.

Q72. What is logged?

```
function App() {
  const [c, setC] = useState(0);
  const click = () => { setC(c + 1); console.log(c); };
  return <button onClick={click}>{c}</button>;
}
```

Logs **0** on the first click. State is a snapshot for that render; the new value appears in the next render.

Q73. Infinite loop: why?

```
const [data, setData] = useState([]);
useEffect(() => { setData([...data, 1]); }, [data]);
```

Each effect creates a new array, which changes `data`, which re-runs the effect. Fix by removing the dependency on its own output, using a functional update with `[]`, or deriving the value instead.

Q74. Why does this object dependency fire every render?

```
const options = { page, size };
useEffect(() => { load(options); }, [options]); // new object every render
```

Depend on primitives `[page, size]`, or `useMemo` the object.

Q75. Does mutating state work?

```
const [list, setList] = useState([]);
const add = () => { list.push(4); setList(list); };
```

No re-render: same reference, `Object.is` says equal. Use `setList(prev => [...prev, 4])`.

10. Machine-Coding / Practical Rounds

Q76. Build a todo list with add, toggle, delete and filter.

```
function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");
  const [filter, setFilter] = useState("all");
  const add = (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos(t => [...t, { id: crypto.randomUUID(), text: text.trim(), done: false }]);
    setText("");
  };
  const toggle = id => setTodos(t => t.map(x => x.id === id ? { ...x, done: !x.done } : x));
  const remove = id => setTodos(t => t.filter(x => x.id !== id));
  const visible = todos.filter(t => filter === "all" || (filter === "done" ? t.done : !t.done));
  return (
    <>
      <form onSubmit={add}><input value={text} onChange={e => setText(e.target.value)} /><button>Add</
    button></form>
      {["all", "active", "done"].map(f => <button key={f} onClick={() => setFilter(f)}>{f}</button>)}
      <ul>{visible.map(t => (
        <li key={t.id}>
          <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
          {t.text} <button onClick={() => remove(t.id)}>x</button>
        </li>))}
      </ul>
    </>
  );
}
```

Note `visible` is derived, not stored in state.

Q77. Build a debounced search with autocomplete that avoids race conditions.

```
function Search() {
  const [q, setQ] = useState("");
  const dq = useDebounce(q, 350);
  const [results, setResults] = useState([]);
  useEffect(() => {
    if (!dq) { setResults([]); return; }
    const c = new AbortController();
    fetch(`/api/merchants?q=${encodeURIComponent(dq)}`, { signal: c.signal })
      .then(r => r.json()).then(setResults).catch(() => {});
    return () => c.abort();
  }, [dq]);
  return (<>
    <input value={q} onChange={e => setQ(e.target.value)} aria-label="Search merchants" />
    <ul>{results.map(m => <li key={m.id}>{m.name}</li>)}</ul>
  </>);
}
```

Discuss: keyboard navigation (ArrowUp/Down, Enter), highlighting matches, caching previous queries, minimum characters.

Q78. Build a paginated table with sorting and server-side paging.

```
function TxnTable() {
  const [page, setPage] = useState(1);
  const [sort, setSort] = useState({ col: "date", dir: "desc" });
  const { data, isLoading } = useQuery({
    queryKey: ["txns", page, sort],
    queryFn: () => api.get("/txns", { params: { page, size: 50, sort: sort.col, dir: sort.dir } }).then(r
    => r.data),
    placeholderData: keepPreviousData
  });
  const toggleSort = col => setSort(s => ({ col, dir: s.col === col && s.dir === "asc" ? "desc" : "asc"
    }));
  if (isLoading) return <Spinner />;
  return (<>
    <table><thead><tr>
      {["date", "rrn", "amount", "status"].map(c => <th key={c} onClick={() => toggleSort(c)}>{c}</th>)}
    </tr></thead>
    <tbody>{data.items.map(t => <tr key={t.id}><td>{t.date}</td><td>{t.rrn}</td><td>{t.amount}</td><td>{t.
    status}</td></tr>)}</tbody></table>
    <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>Prev</button>
    <span>{page} / {data.totalPages}</span>
    <button disabled={page >= data.totalPages} onClick={() => setPage(p => p + 1)}>Next</button>
  </>);
}
```

Q79. Implement infinite scroll.

Use `IntersectionObserver` on a sentinel element at the bottom.

```
function useInfinite(loadMore, hasMore) {
  const ref = useRef(null);
  useEffect(() => {
    if (!hasMore) return;
    const obs = new IntersectionObserver(([e]) => e.isIntersecting && loadMore(), { rootMargin: "200px"
    });
    if (ref.current) obs.observe(ref.current);
    return () => obs.disconnect();
  }, [loadMore, hasMore]);
  return ref;   // <div ref={sentinel} />
}
```

Q80. Build a reusable Modal with portal, ESC to close and focus handling.

```
function Modal({ open, onClose, title, children }) {
  const ref = useRef(null);
  useEffect(() => {
    if (!open) return;
    const prev = document.activeElement;
    ref.current?.focus();
    const onKey = e => e.key === "Escape" && onClose();
    document.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => { document.removeEventListener("keydown", onKey); document.body.style.overflow = "";
    prev?.focus(); };
  }, [open, onClose]);
  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={onClose}>
      <div role="dialog" aria-modal="true" aria-label={title} tabIndex={-1} ref={ref} onClick={e => e.
    stopPropagation()}>
        {children}
      </div>
    </div>, document.body);
}
```

Q81. Build a countdown timer / stopwatch.

```
function Stopwatch() {
  const [ms, setMs] = useState(0);
  const [running, setRunning] = useState(false);
  useEffect(() => {
    if (!running) return;
    const start = Date.now() - ms;
    const id = setInterval(() => setMs(Date.now() - start), 50);
    return () => clearInterval(id);
  }, [running]); // eslint-disable-line
  return (<>
    <h2>{(ms / 1000).toFixed(2)}s</h2>
    <button onClick={() => setRunning(r => !r)}>{running ? "Pause" : "Start"}</button>
    <button onClick={() => { setRunning(false); setMs(0); }}>Reset</button>
  </>);
}
```

Using `Date.now()` instead of counting ticks avoids drift.

Q82. Build a nested (recursive) file explorer / tree.

```
function Tree({ node }) {
  const [open, setOpen] = useState(false);
  if (!node.children) return <li>{node.name}</li>;
  return (
    <li>
      <span onClick={() => setOpen(o => !o)}>{open ? "v" : ">"} {node.name}</span>
      {open && <ul>{node.children.map(c => <Tree key={c.id} node={c} />)}</ul>}
    </li>
  );
}
```

Q83. Implement a multi-step form wizard.

Keep all form data in one parent (`useReducer` or React Hook Form `FormProvider`), render step by index, validate each step before moving next, keep "Back" without losing data, and submit at the end. Persist the draft to sessionStorage for refresh safety.

Q84. Create an OTP input with auto-focus.

```
function Otp({ length = 6, onDone }) {
  const [vals, setVals] = useState(Array(length).fill(""));
  const refs = useRef([]);
  const change = (i, v) => {
    if (!/^\d?$/.test(v)) return;
    const next = [...vals]; next[i] = v; setVals(next);
    if (v && i < length - 1) refs.current[i + 1].focus();
    if (next.every(Boolean)) onDone(next.join(""));
  };
  const key = (i, e) => e.key === "Backspace" && !vals[i] && i > 0 && refs.current[i - 1].focus();
  return vals.map((v, i) => (
    <input key={i} ref={el => (refs.current[i] = el)} value={v} maxLength={1} inputMode="numeric"
           onChange={e => change(i, e.target.value)} onKeyDown={e => key(i, e)} />));
}
```

11. Real-World Scenario Questions

Q85. The dashboard is slow after a filter change. How do you debug it?

Open the React Profiler and record the interaction. Look for: the whole page re-rendering because filter state lives at the top; a context value recreated each render; an expensive computation (grouping 50k rows) in render; unvirtualised list. Fixes: colocate state, memoise derived data with useMemo, move aggregation to the server or a Web Worker, virtualise, and wrap the update in `startTransition` so the input stays responsive.

Q86. Users see old data after updating a record. Why?

The client cache was not invalidated (TanStack Query key not invalidated after mutation), or browser/HTTP caching on GET, or a stale closure in the component. Fix with `invalidateQueries`/`setQueryData` after mutation, correct cache headers, and optimistic update with rollback on error.

Q87. Two quick clicks on "Submit Payment" created two payments. How do you prevent it?

Frontend: disable the button while pending (`isSubmitting`/`isPending`) and ignore re-entry. Backend (the real fix): an **idempotency key** generated per form submission and sent as a header; the API stores it and returns the same result for repeats. Also a unique constraint in DB.

Q88. A component shows the previous user's data briefly after switching users.

State from the previous user persists because the same component instance is reused. Reset with `key={userId}`, clear data on change, and ensure the request for the old user is aborted so its late response can't overwrite the new one.

Q89. Memory leak warning: "Can't perform a React state update on an unmounted component". Fix?

(Older React warning.) A pending async task updates state after unmount. Cancel in cleanup: AbortController for fetch, clearInterval/clearTimeout, unsubscribe from sockets/event listeners.

Q90. How would you implement real-time updates (live transaction feed)?

WebSockets or SignalR (for a .NET backend) or Server-Sent Events. Connect in a custom hook, clean up on unmount, batch incoming messages (buffer and flush every 500ms) to avoid re-rendering for every message, cap the list size, and reconnect with backoff.

```
useEffect(() => {
  const conn = new signalR.HubConnectionBuilder().withUrl("/hubs/txn").withAutomaticReconnect().build();
  conn.on("txn", t => buffer.current.push(t));
  conn.start();
  const id = setInterval(() => { if (buffer.current.length) { const b = buffer.current; buffer.current = [
    ];
      setFeed(f => [...b, ...f].slice(0, 500)); } }, 500);
  return () => { clearInterval(id); conn.stop(); };
}, []);
```

Q91. How do you handle role-based access in the UI?

Load roles/permissions at login (from JWT claims or `/me` endpoint), store in auth context, create `<Can permission="txn.approve">` component or `usePermission` hook to hide/disable UI, and guard routes. The server must still enforce every permission.

Q92. How do you upload a large file (e.g. 200MB settlement file) with progress?

Use `FormData` with axios `onUploadProgress`, or chunked upload (split with `file.slice`, upload parts with an index, server assembles; resume on failure). Validate type/size on client and server, show progress, allow cancel with AbortController.

Q93. How do you handle global errors and show user-friendly messages?

Error Boundary per route for render errors, axios interceptor mapping HTTP errors to toasts (401 -> login, 403 -> "no permission", 500 -> generic + correlation id), and logging to Sentry/Application Insights with the correlation id so backend logs can be matched.

Q94. The bundle is 4MB. What steps do you take?

Analyse with a bundle visualiser; lazy-load routes and heavy libraries (charts, PDF, Excel export); replace heavy libs (moment -> date-fns/dayjs, full lodash -> per-method imports); remove unused polyfills; enable gzip/brotli; split vendor chunks for caching.

Q95. How would you make a component library reusable across multiple apps?

Build in TypeScript with Storybook for documentation, accessibility baked in, theming via CSS variables/design tokens, tree-shakable ESM output, `react` as a peer dependency, semantic versioning, and visual regression tests.

Q96. How do you implement dark mode?

CSS variables for colours, a `data-theme` attribute on `<html>`, default from `prefers-color-scheme`, user choice saved in localStorage, applied before first paint (small inline script) to avoid flashing.

Q97. How do you make a React app accessible?

Semantic HTML (button not div), labels for inputs, keyboard navigation and visible focus, ARIA only where needed, colour contrast, focus management in modals, `alt` text, announcing async status with `aria-live`. Test with axe and a screen reader.

12. Quick-Fire Questions (Short Answers)

Q98. What is StrictMode?

A development-only wrapper that double-invokes renders and effects and warns about unsafe lifecycles to expose bugs. No effect in production.

Q99. What is the difference between element and component?

A component is a function/class. An element is the plain object it returns (`{ type, props }`) describing what to render.

Q100. Can a child component modify props?

No. Props are read-only. The child calls a callback prop to ask the parent to change state.

Q101. What is the purpose of `key` on a Fragment?

When mapping a list that returns multiple elements per item: `<Fragment key={id}><dt/><dd/></Fragment>`.

Q102. What is `children` prop?

Whatever is placed between the opening and closing tag. It enables composition (layout components, cards, providers).

Q103. Synthetic events?

React wraps native events in a cross-browser SyntheticEvent. Since React 17, listeners attach to the root container instead of `document`. Event pooling was removed.

Q104. What is the difference between `onClick={fn}` and `onClick={fn()}`?

The first passes the function; the second calls it during render (and passes its return value), often causing infinite loops when it sets state.

Q105. How do you pass data from child to parent?

Pass a callback prop from the parent; the child calls it with the data.

Q106. What does `useState` return when you call the setter with the same value?

React bails out (no re-render of children) if `Object.is(old, new)` is true, though it may render the component itself once.

Q107. How do you optimise images in React?

Proper sizes and modern formats (WebP/AVIF), `loading="lazy"`, `srcset`, width/height to prevent layout shift, CDN. In Next.js, `next/image`.

Q108. What are environment variables in a React build?

Values embedded at **build time** (`VITE_API_URL`). They are visible in the bundle, so never store secrets in them.

Q109. Why should you not put everything in global state?

Every subscriber may re-render, it couples features, and it makes reasoning harder. Keep state local unless it's genuinely shared.

13. JavaScript Deep Dive and Tricky Output Questions

Q110. What is hoisting? Predict the output.

Declarations are moved to the top of their scope at compile time. `var` is initialised to undefined; function declarations are hoisted with their body; `let/const/class` are hoisted but uninitialised (TDZ).

```
console.log(a); var a = 1;      // undefined
foo(); function foo() { console.log("hi"); }  // hi
bar(); var bar = () => {};      // TypeError: bar is not a function
console.log(b); let b = 2;      // ReferenceError
```

Q111. What is the output: `console.log(typeof null, typeof [], typeof NaN, typeof function(){})`?

`object object number function`. `typeof null === "object"` is a historic bug. Use `Array.isArray` for arrays and `Number.isNaN` for NaN.

Q112. What is `[] + []`, `[] + {}`, `1 + "2"`, `"5" - 2`, `true + 1`?

`""`, `"[object Object]"`, `"12"`, `3`, `2`. `+` concatenates if either side is a string after ToPrimitive; `-` always converts to numbers.

Q113. Why is `0.1 + 0.2 !== 0.3`?

IEEE-754 binary floating point can't represent 0.1 exactly; result is 0.30000000000000004. Compare with tolerance `Math.abs(a-b) < Number.EPSILON`, or store money as integer paise.

Q114. `NaN === NaN`? How to check NaN?

False; NaN is not equal to itself. Use `Number.isNaN(x)` or `Object.is(x, NaN)`. Global `isNaN("abc")` is true because it coerces first.

Q115. What is the difference between `Object.is` and `===`?

Same except `Object.is(NaN, NaN)` is true and `Object.is(0, -0)` is false. React uses `Object.is` to compare state and dependencies.

Q116. Output of this closure-in-loop question with var and an IIFE fix?

```
for (var i = 0; i < 3; i++) { (function (j) { setTimeout(() => console.log(j)); })(i); } // 0 1 2
```

The IIFE creates a new scope per iteration, capturing the current value.

Q117. What are truthy and falsy values?

Falsy: `false, 0, -0, 0n, "", null, undefined, NaN`. Everything else is truthy, including `"0"`, `"false"`, `[]` and `{}`.

Q118. What is the prototype chain?

Every object has an internal `[[Prototype]]`. Property lookup walks up the chain until null. `class` is syntax sugar over prototypes. `Object.create(proto)` creates an object with a given prototype.

Q119. `__proto__` vs `prototype`?

`prototype` is a property on constructor functions used as the prototype for instances created with `new`. `__proto__` (or `Object.getPrototypeOf`) is the actual prototype link of an object.

Q120. What does the `new` keyword do?

Creates an empty object, sets its prototype to `Constructor.prototype`, calls the constructor with `this` bound to it, and returns the object (unless the constructor returns another object).

Q121. call vs apply vs bind.

`call(thisArg, a, b)` invokes immediately with args; `apply(thisArg, [a, b])` invokes with an array; `bind(thisArg)` returns a new function with `this` fixed (and optional partial args).

Q122. Implement Function.prototype.bind polyfill.

```
Function.prototype.myBind = function (ctx, ...pre) {
  const fn = this;
  return function (...args) { return fn.apply(ctx, [...pre, ...args]); };
};
```

Q123. What is currying? Implement `sum(1)(2)(3)`.

Transforming f(a,b,c) into f(a)(b)(c).

```
const curry = fn => function c(...a) { return a.length >= fn.length ? fn(...a) : (...b) => c(...a, ...b);
    };
const add = curry((a, b, c) => a + b + c); add(1)(2)(3); // 6
```

Q124. What is memoisation? Write a memoize function.

```
const memoize = fn => { const cache = new Map();
  return (...args) => { const k = JSON.stringify(args); if (!cache.has(k)) cache.set(k, fn(...args));
    return cache.get(k); }; };
```

Q125. Map vs Object; Set vs Array.

Map keeps insertion order, allows any key type, has `size`, better for frequent add/remove. Object keys are strings/symbols. Set stores unique values with O(1) `has`; use it to dedupe: `[...new Set(arr)]`.

Q126. WeakMap / WeakSet - why use them?

Keys are weakly held (objects only), so entries are garbage-collected when the key has no other references. Useful for caching metadata on DOM nodes or objects without leaks. Not iterable.

Q127. What are Symbols?

Unique, immutable primitive values used as non-colliding property keys; well-known symbols customise behaviour (`Symbol.iterator`).

Q128. What are generators and iterators?

An iterator has `next()` returning `{value, done}`. Generator functions (`function*`) pause at `yield`. Used for lazy sequences; Redux-Saga is built on generators.

Q129. Explain async/await error handling and parallel calls.

Wrap in try/catch. `await` in sequence runs calls one after another; for independence use `await Promise.all([a(), b()])`. Common mistake: `forEach(async ...)` doesn't wait; use `for...of` or `Promise.all(arr.map(...))`.

Q130. Output: async ordering puzzle.

```
async function f() { console.log(1); await null; console.log(2); }
console.log(3); f(); console.log(4);
// 3 1 4 2
```

Code before the first await runs synchronously; the rest is a microtask.

Q131. Implement Promise.all polyfill.

```
function all(ps) { return new Promise((res, rej) => { const out = []; let done = 0;
  if (!ps.length) res(out);
  ps.forEach((p, i) => Promise.resolve(p).then(v => { out[i] = v; if (++done === ps.length) res(out); },
    rej)); }); }
```

Q132. What are the Promise states?

Pending, fulfilled, rejected. Once settled it can't change. `.then` callbacks run as microtasks.

Q133. Spread vs rest operator.

Same `...` syntax: spread expands an iterable/object (`[...a, ...b]`, `{...o}`); rest collects remaining items (`function f(...args)`, `const {a, ...others} = o`).

Q134. Destructuring with defaults and renaming.

`const { name: userName = "Guest", address: { city } = {} } = user; const [first, , third] = arr;`

Q135. Optional chaining pitfalls.

`a?.b.c` still throws if `a.b` is undefined; use `a?.b?.c`. Function call: `fn?.()`. Can't be used on the left side of an assignment.

Q136. map vs forEach vs filter vs reduce.

`map` returns a new same-length array; `forEach` returns undefined (side effects); `filter` returns elements passing a test; `reduce` folds into one value. In JSX use `map` to render lists.

Q137. Implement Array.prototype.map and reduce polyfills.

```
Array.prototype.myMap = function (cb) { const r = []; for (let i = 0; i < this.length; i++) if (i in
    this) r[i] = cb(this[i], i, this); return r; };
Array.prototype.myReduce = function (cb, init) { let i = 0, acc = init;
  if (arguments.length < 2) { acc = this[0]; i = 1; }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this); return acc; };
```

Q138. Flatten a nested array without `flat`.

```
const flatten = a => a.reduce((acc, x) => Array.isArray(x) ? acc.concat(flatten(x)) : acc.concat(x), []);
```

Q139. Deep clone an object (handle arrays, dates, circular refs).

`structuredClone(obj)` handles most types and cycles (not functions/DOM). Manual: recursion with a WeakMap for visited objects. `JSON.parse(JSON.stringify())` loses dates, undefined, functions, Map/Set.

Q140. Implement deep equality check.

```
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== "object" || typeof b !== "object" || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every(k => deepEqual(a[k], b[k]));
}
```

Q141. Event bubbling vs capturing vs delegation.

Events capture from window down to the target, then bubble up. `stopPropagation` stops it. Delegation attaches one listener to a parent and uses `event.target` (React effectively delegates at the root).

Q142. preventDefault vs stopPropagation.

`preventDefault` stops the browser default action (form submit, link navigation). `stopPropagation` stops the event reaching other handlers up/down the tree.

Q143. localStorage vs sessionStorage vs cookies vs IndexedDB.

localStorage: ~5MB, persists, same origin, synchronous. sessionStorage: per tab, cleared on close. Cookies: ~4KB, sent with every request, can be HttpOnly. IndexedDB: large async structured storage for offline apps.

Q144. ES modules vs CommonJS.

ESM (`import/export`): static, tree-shakable, async loading, live bindings. CommonJS (`require/module.exports`): dynamic, synchronous, Node's legacy format.

Q145. Named vs default export.

A module can have many named exports and one default. Named exports enforce consistent names and are better for tree-shaking and refactoring.

Q146. What is strict mode?

`"use strict"` (automatic in modules and classes) disallows silent errors: undeclared variables throw, `this` is undefined in plain functions, duplicate params are errors.

Q147. What is the output of `[1,2,3].map(parseInt)`?

`[1, NaN, NaN]` because map passes (value, index): `parseInt("2", 1)` and `parseInt("3", 2)` are invalid radixes/digits.

Q148. What is the output of `console.log(0 || "a", 0 ?? "a", "" && "b")`?

`a 0 ""`. `||` falls back on any falsy, `??` only on null/undefined, `&&` returns the first falsy or the last value.

Q149. What is a pure function and why does React care?

Same input gives same output, no side effects. React components and reducers must render purely so React can re-run, pause or discard renders safely.

Q150. What is immutability and how do you update nested arrays immutably?

```
setOrders(os => os.map(o => o.id === id ? { ...o, items: o.items.filter(i => i.sku !== sku) } : o));
```

Avoid `push`, `splice`, `sort`, `reverse` on state; use `toSorted`, `toReversed`, `with` (ES2023) or copies.

Q151. What are template literals and tagged templates?

Backtick strings with `${}` interpolation and multi-line. Tagged templates `tag\...\` let a function process parts (used by styled-components and GraphQL gql`).

Q152. What is the difference between `for...in` and `for...of`?

`for...in` iterates enumerable keys (including inherited); `for...of` iterates values of iterables (arrays, strings, Map, Set).

Q153. What are getters/setters and Object.freeze?

Accessors compute values on property access. `Object.freeze` makes an object shallowly immutable; nested objects still mutable.

Q154. What is the difference between `slice` and `splice`?

`slice(start, end)` returns a copy without changing the array. `splice` mutates (removes/inserts) and returns removed items.

Q155. What is short-circuit evaluation used for in React?

Conditional rendering (`cond && <X/>`) and defaults (`value ?? "-"`). Beware rendering `0`.

Q156. What is a polyfill vs transpiler?

Transpiler (Babel, SWC, esbuild) converts new syntax to older syntax; polyfill adds missing APIs at runtime (e.g. `Promise`, `Array.prototype.includes`).

Q157. Implement an event emitter.

```
class Emitter { #m = new Map();
  on(e, f) { (this.#m.get(e) ?? this.#m.set(e, new Set()).get(e)).add(f); return () => this.off(e, f); }
  off(e, f) { this.#m.get(e)?.delete(f); }
  emit(e, ...a) { this.#m.get(e)?.forEach(f => f(...a)); } }
```

Q158. What are private class fields?

`#field` is truly private (enforced by the engine), unlike the `_field` convention.

Q159. What is the difference between `setTimeout(fn, 0)` and `requestAnimationFrame`?

setTimeout schedules a macrotask (min ~4ms when nested). rAF runs right before the next repaint, ideal for animations.

Q160. What is a Web Worker?

A background thread for CPU-heavy work (parsing large CSV, crypto) that communicates via `postMessage`; no DOM access.

Q161. What is CORS preflight?

For non-simple requests (custom headers, PUT/DELETE, JSON content type) the browser sends an OPTIONS request first; the server must reply with matching `Access-Control-Allow-*` headers.

14. React Core: More Concepts and Tricky Questions

Q162. Is React a library or framework? Why does it matter?

Library: it handles the view layer. You choose routing, data, forms. Frameworks like Next.js or Remix add conventions, routing, SSR and data loading.

Q163. What is declarative vs imperative UI?

Declarative: describe the UI for each state (`{loading ? <Spinner/> : <Table/>}`). Imperative: manually manipulate DOM (`el.style.display = "none"`). Declarative is easier to reason about.

Q164. What does `ReactDOM.createRoot` do vs `ReactDOM.render`?

`createRoot` (React 18) enables concurrent features and automatic batching. `render` is legacy (removed in React 19).

Q165. Can a component return null, a string, an array?

Yes: `null` renders nothing, strings/numbers render text, arrays render lists (need keys). `undefined` is allowed since React 18.

Q166. Why must component names start with a capital letter?

JSX treats lowercase tags as DOM elements (`<div>`) and capitalised ones as components. `<myComp/>` would try to create an unknown HTML element.

Q167. What is `className` and `htmlFor`?

JSX uses DOM property names because `class` and `for` are reserved words in JS.

Q168. How do inline styles work in React?

Object with camelCased properties: `style={{ backgroundColor: "red", marginTop: 8 }}` (numbers become px for most properties).

Q169. What are the styling options in React?

Plain CSS, CSS Modules (scoped class names), Sass, Tailwind CSS, CSS-in-JS (styled-components, Emotion), component libraries (MUI, Ant Design, shadcn/ui). Zero-runtime options are preferred with Server Components.

Q170. What is the default prop pattern in function components?

Default parameters in destructuring: `function Btn({ size = "md" })`. `defaultProps` on function components is deprecated.

Q171. What is PropTypes?

Runtime prop validation for JS projects (dev only). Mostly replaced by TypeScript; removed from React 19's core checks.

Q172. What is the `children` type when you pass multiple children?

An array. Use `React.Children.map/count/toArray` to work with it safely, or better pass explicit props.

Q173. What is `React.cloneElement` and when to use it?

Clones an element with new props (e.g. injecting `onClick` into children of a compound component). Prefer context or render props; cloneElement is fragile.

Q174. What is `dangerouslySetInnerHTML`?

Sets raw HTML: `<div dangerouslySetInnerHTML={{ __html: clean }} />`. Must sanitise (DOMPurify) to prevent XSS.

Q175. How do you render HTML entities or special characters in JSX?

Use the literal character, `{"\u00A9"}`, or `©` directly in JSX text.

Q176. What happens if you call a hook conditionally? Show the error scenario.

```
if (isAdmin) { const [x, setX] = useState(0); } // breaks hook order
```

React reports "Rendered more hooks than during the previous render." Move the condition inside the hook or split components.

Q177. What is the difference between rendering and committing?

Render: React calls components to compute the new tree (pure, may happen many times). Commit: React applies changes to the DOM and runs effects. A render doesn't always lead to DOM changes.

Q178. Does a parent re-render re-mount children?

No, it re-renders them (same instances, state preserved) unless the element type or key changes, or position in the tree changes.

Q179. Why does defining a component inside another component cause bugs?

```
function Parent() { function Child() { ... } return <Child />; }
```

`Child` is a new function each render, so React sees a different type, unmounting and remounting it, losing state and focus. Define components at module level.

Q180. When is state preserved or reset?

State is tied to a position in the tree with the same type. `{isA ? <Counter/> : <Counter/>}` keeps state (same position, same type). Give different keys to reset.

Q181. What is "derived state" and why avoid copying props into state?

```
const [name, setName] = useState(props.name); // won't update when props.name changes
```

Compute from props during render, or use a key to reset. Copying props into state creates two sources of truth.

Q182. What is the `key` prop's scope?

Keys must be unique among siblings only, not globally. They're not passed as a prop to the component.

Q183. What happens with duplicate keys?

React warns and may mix up components, causing duplicated or omitted items.

Q184. Explain the component lifecycle in function components.

Mount: render, commit, `useLayoutEffect`, paint, `useEffect`. Update: render, commit, cleanup of previous effects, run new effects. Unmount: cleanups run.

Q185. Class lifecycle methods and their hook equivalents.

`componentDidMount` -> `useEffect(..., [])`; `componentDidUpdate` -> `useEffect` with deps; `componentWillUnmount` -> cleanup; `shouldComponentUpdate` -> `React.memo`; `getDerivedStateFromProps` -> compute during render; `getSnapshotBeforeUpdate` -> no direct hook (useLayoutEffect + ref).

Q186. What is `shouldComponentUpdate` vs `PureComponent`?

`shouldComponentUpdate(nextProps, nextState)` returns false to skip rendering. `PureComponent` implements it with shallow comparison.

Q187. What is the output? Clicking a button inside a form.

```
<form onSubmit={save}><button onClick={log}>Save</button></form>
```

`log` runs, then the form submits (button default type is "submit") and the page reloads unless `e.preventDefault()` in `save`. Use `type="button"` for non-submit buttons.

Q188. How do you handle multiple inputs with one change handler?

```
const onChange = e => setForm(f => ({ ...f, [e.target.name]: e.target.type === "checkbox" ? e.target.
    checked : e.target.value }));
```

Q189. Why is input value showing "A component is changing an uncontrolled input to be controlled"?

`value` started as `undefined` then became a string. Initialise state with `""` instead of undefined/null.

Q190. How do you handle file inputs in React?

File inputs are always uncontrolled; read `e.target.files[0]` and upload with FormData. To reset, change its key or clear via ref.

Q191. Can you use async functions as components or effects?

Client components cannot be async (Server Components can). `useEffect(async () => {})` is wrong because it returns a promise instead of a cleanup; define an async function inside and call it.

Q192. What is the difference between `useEffect` cleanup and unmount?

Cleanup runs before every re-run of the effect AND on unmount; not only on unmount.

Q193. What are synthetic event handler naming conventions?

Props: `onSomething`, handlers: `handleSomething`. Events like `onChange` in React fire on every keystroke (unlike native `change`).

Q194. How to pass arguments to an event handler?

`onClick={() => remove(id)}` or `data-id` attributes read from `e.currentTarget.dataset.id` (avoids new closures per row).

Q195. e.target vs e.currentTarget?

`target` is the element that triggered the event; `currentTarget` is the element whose handler is running.

Q196. What is the React DevTools "Components" tab used for?

Inspect component tree, props, state, hooks and context values, and locate the source. The Profiler tab records renders and their reasons.

Q197. What is React Strict DOM / React Native - how is React cross-platform?

React's reconciler is renderer-agnostic: react-dom for web, React Native for mobile, react-three-fiber for 3D. Same component model, different host components.

15. Hooks: Tricky and Advanced Questions

Q198. What is the output on mount in StrictMode (development)?

```
useEffect(() => { console.log("mount"); return () => console.log("cleanup"); }, []);
// mount, cleanup, mount
```

Q199. Do state setters change identity between renders?

No, `setState` and `dispatch` are stable; you can omit them from dependency arrays.

Q200. Can useState hold a function?

Yes, but you must wrap it: `setFn(() => myFn)`, because passing a function directly is treated as an updater.

Q201. Why doesn't my useEffect pick up the latest prop inside setInterval?

Stale closure; deps array missing the prop. Add it to deps, or store it in a ref updated every render, or use the `useEffectEvent` pattern (experimental) for non-reactive logic.

Q202. What is the "latest ref" pattern?

```
const cbRef = useRef(cb);
useLayoutEffect(() => { cbRef.current = cb; });
useEffect(() => { const id = setInterval(() => cbRef.current(), 1000); return () => clearInterval(id); },
    []);
```

The interval stays stable but always calls the newest callback (basis of `useInterval`).

Q203. Does updating a ref cause a re-render?

No. Refs are mutable containers; changing `.current` is invisible to React. Don't read/write refs during render (except lazy init).

Q204. Callback refs - what are they?

A function passed as `ref` is called with the node on mount and null on unmount (React 19 allows returning cleanup). Useful to measure a node when it appears.

```
const measure = useCallback(node => { if (node) setH(node.getBoundingClientRect().height); }, []);
```

Q205. Can you use a ref to store previous state?

Yes, via an effect that assigns after render (see `usePrevious`). Or store previous value in state and compare during render.

Q206. When would useMemo actually make things slower?

For cheap computations: the dependency comparison and memory cost exceed the savings. Also memoising when deps change every render does nothing.

Q207. Is useMemo a semantic guarantee?

No. React may discard cached values (e.g. offscreen). Use it as a performance hint, not for correctness.

Q208. useCallback without React.memo on the child - does it help?

Usually not. The child re-renders anyway because the parent re-rendered. useCallback helps when the function is a dependency of an effect or the child is memoised.

Q209. How do you write a useLocalStorage hook?

```
function useLocalStorage(key, init) {
  const [v, setV] = useState(() => { try { const s = localStorage.getItem(key); return s ? JSON.parse(s) :
     init; } catch { return init; } });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(v)); }, [key, v]);
  return [v, setV];
}
```

Q210. Write useOnClickOutside.

```
function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const fn = e => { if (ref.current && !ref.current.contains(e.target)) handler(e); };
    document.addEventListener("mousedown", fn); return () => document.removeEventListener("mousedown",
    fn);
  }, [ref, handler]);
}
```

Q211. Write useWindowSize.

```
function useWindowSize() {
  const [s, setS] = useState({ w: window.innerWidth, h: window.innerHeight });
  useEffect(() => { const f = () => setS({ w: window.innerWidth, h: window.innerHeight });
    window.addEventListener("resize", f); return () => window.removeEventListener("resize", f); }, []);
  return s;
}
```

Q212. Write useToggle and useCounter.

```
const useToggle = (init = false) => { const [v, s] = useState(init); return [v, useCallback(() => s(x =>
    !x), [])]; };
```

Q213. Write useInterval (Dan Abramov style).

Store the callback in a ref updated on every render; set the interval in an effect depending only on `delay`; pass `null` delay to pause.

Q214. Write useMediaQuery.

```
function useMediaQuery(q) {
  return useSyncExternalStore(cb => { const m = matchMedia(q); m.addEventListener("change", cb); return (
    ) => m.removeEventListener("change", cb); },
    () => matchMedia(q).matches, () => false);
}
```

Q215. How do you share state between two components using a custom hook?

You can't: each hook call has its own state. To share, lift state up, use context, or an external store.

Q216. What is useDebugValue?

Labels custom hooks in React DevTools: `useDebugValue(isOnline ? "Online" : "Offline")`.

Q217. What is useInsertionEffect?

Runs before layout effects; for CSS-in-JS libraries to inject styles before reading layout. Not for app code.

Q218. What happens if dependencies include an object created each render?

The effect runs every render. Depend on primitive fields or memoise the object.

Q219. Why does ESLint's exhaustive-deps rule matter?

Missing dependencies cause stale values and bugs. Instead of disabling the rule, restructure: move functions inside the effect, use functional updates, or refs.

Q220. How do you fetch data on button click instead of mount?

Call an async function in the event handler; no effect needed. Effects are for synchronising with props/state, not responding to user events.

Q221. How to run an effect only on updates, not on mount?

```
const first = useRef(true);
useEffect(() => { if (first.current) { first.current = false; return; } save(value); }, [value]);
```

(StrictMode double-invoke may need care.) Often a sign the logic belongs in the event handler.

Q222. What is useOptimistic (React 19)?

Shows an optimistic value while an async action is pending, reverting automatically if it fails.

```
const [optimistic, addOptimistic] = useOptimistic(messages, (state, msg) => [...state, { ...msg, sending:
    true }]);
```

Q223. What is useActionState?

Wraps a form action, returning `[state, formAction, isPending]`, so you get the last result and pending status without manual state.

Q224. What is the `use` hook?

Reads a promise (suspends until resolved) or context. Unlike other hooks it can be called inside conditions and loops.

Q225. What is useFormStatus?

Called inside a form's child component to get `{ pending, data, method }` of the parent form submission (e.g. disable the submit button).

Q226. What is the React Compiler?

A build-time compiler that automatically memoises components and values, making most manual useMemo/useCallback/memo unnecessary. Requires following the Rules of React.

Q227. What is useEffectEvent?

An API (stabilised in React 19.2) to extract non-reactive logic from effects: the event function always sees latest props/state but doesn't need to be a dependency.

16. Rendering, Reconciliation and Performance (Advanced)

Q228. What is the diffing algorithm's assumption about element types?

Two elements of different types produce different trees; React does not try to diff `<div>` into `<span>`, it replaces the subtree.

Q229. Why is reconciliation O(n) instead of O(n^3)?

Generic tree-diff is O(n^3). React uses heuristics (type comparison, keys, level-by-level) to get O(n).

Q230. What is batching? Give an example of what React 18 changed.

Grouping multiple state updates into one re-render. Before 18, updates inside setTimeout/promises were not batched; now they are. Opt out with `flushSync`.

Q231. What does flushSync do?

Forces React to apply the update synchronously (e.g. to scroll to a newly added item immediately). Hurts performance; use rarely.

Q232. What is concurrent rendering?

React can prepare multiple versions of the UI, interrupt low-priority renders for urgent ones (typing), and avoid blocking the main thread.

Q233. What are lanes / priorities?

Fiber's internal priority model: discrete input (click, typing) is high priority; transitions are lower; idle work lowest.

Q234. What is "tearing"?

In concurrent rendering, different components reading an external store could see different values in the same render. `useSyncExternalStore` prevents it.

Q235. What is time slicing?

Splitting rendering work into chunks and yielding to the browser between chunks so the page stays responsive.

Q236. How does React.memo compare props? Can you customise it?

Shallow comparison with `Object.is` per prop. Pass a custom comparator: `React.memo(Comp, (prev, next) => prev.id === next.id)` (return true to skip render).

Q237. Why might React.memo not prevent re-render when context changes?

Context consumers re-render regardless of memo when the context value changes.

Q238. What is the "children as props" optimisation?

```
function Scroller({ children }) { const [y, setY] = useState(0); ... return <div onScroll={...}>{
    children}</div>; }
<Scroller><ExpensiveTree /></Scroller>
```

`ExpensiveTree` is created by the parent, so it doesn't re-render when Scroller's state changes.

Q239. What is state colocation?

Keeping state in the lowest component that needs it to limit re-render scope.

Q240. How do you prevent a context re-rendering everything?

Split into multiple contexts, memoise the value, separate state and dispatch contexts, or use a selector-based store. Libraries like `use-context-selector` enable selecting slices.

Q241. How do you identify why a component re-rendered?

Profiler "Record why each component rendered" setting, or why-did-you-render library, or logging in render.

Q242. What is lazy loading of images and components?

Images: `loading="lazy"` / IntersectionObserver. Components: `React.lazy` + Suspense. Also preload on hover for perceived speed.

Q243. What is prefetching/preloading in routing?

Loading code or data before navigation (on hover or idle). React Router loaders and Next.js `<Link>` prefetch automatically.

Q244. What are Web Vitals (LCP, INP, CLS)?

LCP: time to render the largest content (target < 2.5s). INP: responsiveness to interactions (< 200ms). CLS: visual stability (< 0.1). Measure with the `web-vitals` library and Lighthouse.

Q245. How do you reduce CLS in React?

Reserve space for images and ads (width/height or aspect-ratio), skeleton loaders of the right size, avoid inserting content above existing content.

Q246. How do you improve INP?

Keep event handlers light, use `startTransition` for heavy updates, debounce, virtualise lists, move work to Web Workers.

Q247. What is a skeleton loader vs spinner?

Skeleton shows placeholder shapes of the final layout, reducing perceived wait and layout shift.

Q248. What is tree shaking and what breaks it?

Removing unused exports at build time. Broken by CommonJS modules, side-effectful imports, and importing whole libraries (`import _ from "lodash"`).

Q249. How do you analyse bundle size?

`vite-bundle-visualizer`, `source-map-explorer`, `webpack-bundle-analyzer`. Check duplicate packages and large dependencies.

Q250. What is memoisation of selectors in Redux, and why does returning a new array in useSelector cause re-renders?

`useSelector(s => s.items.filter(...))` returns a new array each time, failing reference equality, so the component re-renders on every store change. Use `createSelector` or `shallowEqual`.

Q251. How do you handle expensive calculations on large datasets in the browser?

useMemo for caching, Web Workers to offload, chunking with `requestIdleCallback`, or move aggregation to the server.

Q252. What is windowing vs pagination vs infinite scroll?

Windowing renders only visible rows of a large loaded list. Pagination loads one page at a time. Infinite scroll appends pages as you scroll (often combined with windowing).

Q253. Why are inline functions in JSX not always bad?

Creating functions is cheap; it only matters when passed to memoised children or used as effect deps.

Q254. What is hydration cost and how do Server Components help?

Hydration downloads and runs JS for every client component. Server Components send no JS for themselves, reducing bundle and hydration time.

Q255. What is the Offscreen/Activity API?

`<Activity mode="hidden">` (React 19.2) keeps a subtree mounted but hidden, preserving state and deprioritising updates, like keeping tabs alive.

Q256. How to avoid layout thrashing?

Batch DOM reads and writes; don't interleave `offsetHeight` reads with style writes in loops; use `useLayoutEffect` or rAF.

Q257. How would you optimise a form with 200 fields?

Uncontrolled inputs (React Hook Form), field-level subscriptions, split into sections/steps, memoise sections, avoid storing the whole form in a top-level state updated on every keystroke.

17. State Management (Advanced)

Q258. What is the difference between client state and server state?

Client state: UI-owned (modal open, filters, theme). Server state: remote data owned by the backend; can be stale, needs caching, refetch, sync. Different tools suit each.

Q259. Explain Redux core principles.

Single source of truth (one store), state is read-only (change only via actions), changes made with pure reducers.

Q260. What is middleware in Redux?

A function wrapping dispatch: `store => next => action => {...}`. Used for logging, async (thunk), analytics, error reporting.

```
const logger = store => next => action => { console.log(action.type); const r = next(action); console.log(
    store.getState()); return r; };
```

Q261. Redux Thunk vs Redux Saga vs RTK Query.

Thunk: dispatch functions for async; simple. Saga: generator-based side effects, good for complex flows (cancellation, debouncing, race). RTK Query: data fetching and caching built on Redux; removes most thunks.

Q262. Show an RTK Query API slice.

```
export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: "/api", prepareHeaders: (h, { getState }) => { h.set(
    "Authorization", `Bearer ${getState().auth.token}`); return h; } }),
  tagTypes: ["Txn"],
  endpoints: b => ({
    getTxns: b.query({ query: date => `txns?date=${date}`, providesTags: ["Txn"] }),
    approve: b.mutation({ query: id => ({ url: `txns/${id}/approve`, method: "POST" }), invalidatesTags: [
    "Txn"] })
  })
});
export const { useGetTxnsQuery, useApproveMutation } = api;
```

Q263. What is normalised state? Why use createEntityAdapter?

Storing entities by ID (`{ ids: [], entities: {} }`) avoids duplication and makes updates O(1). `createEntityAdapter` generates reducers and selectors for that shape.

Q264. How do you persist Redux state?

redux-persist (localStorage/IndexedDB), whitelist only needed slices, version and migrate state, never persist sensitive tokens in plain storage.

Q265. What is configureStore doing for you?

Combines reducers, adds thunk middleware, Redux DevTools, and dev checks for mutation and non-serialisable values.

Q266. Why should actions and state be serialisable?

Enables DevTools time travel, persistence and predictable debugging. Don't store Promises, class instances or functions in the store.

Q267. How does Zustand work? Example.

```
const useCart = create(set => ({ items: [], add: i => set(s => ({ items: [...s.items, i] })), clear: ()
    => set({ items: [] }) }));
const count = useCart(s => s.items.length);  // re-renders only when length changes
```

Q268. What is Jotai / Recoil (atomic state)?

State split into small atoms; components subscribe to atoms they use. Derived atoms compute from others. Fine-grained updates without a single store.

Q269. What is MobX?

Observable state with automatic dependency tracking; components re-render when observed values change. Mutable style, less boilerplate.

Q270. Context API: what is the default value used for?

Used when a component has no matching Provider above it (useful for tests or sensible defaults).

Q271. Can you have nested providers of the same context?

Yes; the nearest provider wins. Useful for theming sections differently.

Q272. How do you structure auth state in React?

AuthProvider holding user and token status, `login/logout` functions, loading state for initial session check (call `/me` on startup), and an axios interceptor that reads the token. Hide UI by roles via a permission hook.

Q273. How do you sync state with the URL?

Store filters, page and sort in query params (`useSearchParams`) so links are shareable and back/forward works.

Q274. TanStack Query: staleTime vs gcTime (cacheTime).

`staleTime`: how long data is fresh (no refetch). `gcTime`: how long unused cached data is kept in memory before garbage collection.

Q275. TanStack Query: how do optimistic updates work?

```
useMutation({ mutationFn: approve,
  onMutate: async id => { await qc.cancelQueries({ queryKey: ["txns"] }); const prev = qc.getQueryData([
    "txns"]);
    qc.setQueryData(["txns"], old => old.map(t => t.id === id ? { ...t, status: "APPROVED" } : t));
    return { prev }; },
  onError: (_e, _id, ctx) => qc.setQueryData(["txns"], ctx.prev),
  onSettled: () => qc.invalidateQueries({ queryKey: ["txns"] }) });
```

Q276. What are dependent and parallel queries?

Dependent: `enabled: !!userId` waits for prerequisite data. Parallel: multiple `useQuery` calls or `useQueries` for a dynamic list.

Q277. How does useInfiniteQuery work?

Fetches pages with `getNextPageParam`; data is `{ pages, pageParams }`; call `fetchNextPage` when the sentinel is visible.

Q278. SWR vs TanStack Query?

SWR is lighter with stale-while-revalidate basics; TanStack Query has richer mutations, devtools, infinite queries and cache control.

Q279. When would you use useReducer + Context instead of Redux?

Small to medium apps with moderate shared state, no need for middleware/devtools ecosystem.

18. Routing, Forms and API Integration (More)

Q280. Client-side routing vs server-side routing.

Client-side: JS intercepts navigation and renders views without full page reload (History API). Server-side: each URL is a new request returning HTML.

Q281. BrowserRouter vs HashRouter vs MemoryRouter.

BrowserRouter uses clean URLs (server must serve index.html for all routes). HashRouter uses `#/path` (works on static hosts without config). MemoryRouter keeps history in memory (tests, React Native).

Q282. Why does refreshing a deep route give 404 in production?

The server looks for a file at `/reports/2` that doesn't exist. Configure fallback to `index.html` (IIS rewrite rule, Nginx `try_files`).

Q283. How do you read query parameters?

```
const [params, setParams] = useSearchParams(); const page = Number(params.get("page") ?? 1);
```

Q284. How do you programmatically navigate?

```
const navigate = useNavigate(); navigate("/txn/5", { replace: true, state: { from: "list" } }); navigate(-1);
```

Q285. What are nested routes and Outlet?

Child routes render inside the parent's `<Outlet/>`, enabling shared layouts (sidebar + header) with changing content.

Q286. What are React Router loaders and actions (data routers)?

`createBrowserRouter` routes can define `loader` (fetch before render) and `action` (handle form submissions), reducing waterfalls and loading state code.

Q287. How do you block navigation with unsaved changes?

`useBlocker` in data routers (or `beforeunload` for browser close) to show a confirmation dialog.

Q288. How do you implement lazy routes?

`{ path: "reports", lazy: () => import("./Reports") }` or `React.lazy` inside `element`.

Q289. How do you implement breadcrumbs?

Route `handle` metadata plus `useMatches()` to build crumbs from matched routes.

Q290. How do you handle 404 and error routes?

A catch-all `path="*"` route and `errorElement` on routes to render errors thrown from loaders/components.

Q291. Formik vs React Hook Form.

Formik uses controlled inputs and re-renders on every keystroke; RHF uses refs (uncontrolled), fewer re-renders, smaller bundle. RHF is the current default.

Q292. How do you integrate a controlled UI-library input with React Hook Form?

Use `Controller`:

```
<Controller name="date" control={control} rules={{ required: true }}
  render={({ field }) => <DatePicker selected={field.value} onChange={field.onChange} />} />
```

Q293. How do you build dynamic form fields (add/remove rows)?

`useFieldArray` in RHF: `fields.map((f, i) => <input key={f.id} {...register(\items.${i}.amount\)} />)`, plus `append`/`remove`.

Q294. How do you show server-side validation errors in a form?

Map API error response (field -> message) using `setError("rrn", { message })`, and show a general error for non-field errors.

Q295. How do you handle file download from an API?

```
const res = await api.get(`/reports/${id}`, { responseType: "blob" });
const url = URL.createObjectURL(res.data); const a = document.createElement("a");
a.href = url; a.download = "report.xlsx"; a.click(); URL.revokeObjectURL(url);
```

Q296. fetch vs axios.

fetch is built in but doesn't reject on HTTP errors (check `res.ok`) and lacks interceptors and upload progress. axios has interceptors, automatic JSON, timeouts, progress events and request cancellation.

Q297. How do you cancel an axios request?

Pass `signal: controller.signal` (AbortController); call `controller.abort()`.

Q298. How do you handle API errors centrally?

Axios response interceptor normalises errors into `{ status, message, fieldErrors }`, shows toasts, redirects on 401, and rethrows for components that need it.

Q299. How do you poll an API?

TanStack Query `refetchInterval`, or setInterval in an effect with cleanup. Stop polling when the tab is hidden (`document.visibilityState`).

Q300. What is long polling vs SSE vs WebSockets?

Long polling: request held open until data. SSE: one-way server push over HTTP (text). WebSockets: full-duplex. Choose SSE for notifications, WebSockets/SignalR for bi-directional real-time.

Q301. How do you handle timezones and dates in React?

Store UTC in backend, format in the user's timezone with `Intl.DateTimeFormat` or date-fns-tz/dayjs. Beware `new Date("2026-01-05")` is parsed as UTC.

Q302. How do you format currency in React?

`new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" }).format(amount)`.

Q303. How do you implement i18n?

react-i18next / FormatJS: translation JSON per locale, `t("key")`, plurals, interpolation, lazy-loaded namespaces, RTL support.

19. TypeScript with React

Q304. How do you type children?

`children: React.ReactNode` for anything renderable. `React.PropsWithChildren<Props>` adds it.

Q305. ReactNode vs ReactElement vs JSX.Element.

ReactNode: anything renderable (string, number, null, element, array). ReactElement/JSX.Element: an element object only.

Q306. How do you type useState with null initial value?

`const [user, setUser] = useState<User | null>(null);`

Q307. How do you type a ref to a DOM element?

`const inputRef = useRef<HTMLInputElement>(null);`

Q308. How do you type event handlers?

`(e: React.ChangeEvent<HTMLInputElement>) => void`, `React.FormEvent<HTMLFormElement>`, `React.MouseEvent<HTMLButtonElement>`.

Q309. How do you extend native element props?

```
type ButtonProps = React.ComponentPropsWithoutRef<"button"> & { variant?: "primary" | "ghost" };
const Button = ({ variant = "primary", ...rest }: ButtonProps) => <button className={variant} {...rest} /
    >;
```

Q310. How do you type a generic component (e.g. Table<T>)?

```
type TableProps<T> = { rows: T[]; columns: { key: keyof T; header: string }[]; getId: (r: T) => string };
function Table<T>({ rows, columns, getId }: TableProps<T>) { ... }
```

Q311. interface vs type in TS?

Both describe object shapes. Interfaces merge declarations and are extendable; types can express unions, intersections, mapped and conditional types. Pick one convention.

Q312. What are utility types you use often?

`Partial`, `Required`, `Pick`, `Omit`, `Record`, `Readonly`, `ReturnType`, `Parameters`, `NonNullable`, `Awaited`.

Q313. What is a discriminated union? Example with API state.

```
type State<T> = { status: "idle" } | { status: "loading" } | { status: "success"; data: T } | { status:
    "error"; error: string };
if (s.status === "success") s.data; // narrowed
```

Q314. How do you type context safely?

```
const AuthCtx = createContext<AuthValue | undefined>(undefined);
export const useAuth = () => { const c = useContext(AuthCtx); if (!c) throw new Error("useAuth outside
    AuthProvider"); return c; };
```

Q315. How do you type a custom hook returning a tuple?

Use `as const`: `return [value, toggle] as const;` so TS infers `readonly [boolean, () => void]` not `(boolean | fn)[]`.

Q316. What is `unknown` vs `any`?

`any` disables type checking. `unknown` must be narrowed before use; prefer it for API responses or caught errors.

Q317. How do you type API responses safely at runtime?

TS types vanish at runtime; validate with Zod (`schema.parse(data)`) and infer types with `z.infer<typeof schema>`.

Q318. What is `satisfies`?

Checks a value matches a type while keeping its narrower inferred type: `const routes = {...} satisfies Record<string, Route>`.

Q319. What are enums vs union string literals?

Union literals (`"PENDING" | "MATCHED"`) are lighter and erase at runtime; enums generate code. Many teams prefer unions or `as const` objects.

Q320. How do you type forwardRef components?

`const Input = forwardRef<HTMLInputElement, InputProps>((props, ref) => <input ref={ref} {...props} />);` In React 19 just accept `ref` in props.

Q321. What are keyof and typeof in TS?

`keyof T` gives a union of property names; `typeof value` gets the type of a value; `(typeof arr)[number]` gives element type.

Q322. What are generics constraints in TS?

`function getProp<T, K extends keyof T>(o: T, k: K): T[K]`.

Q323. What is strict mode in tsconfig and why enable it?

Enables `strictNullChecks`, `noImplicitAny` and more, catching null bugs at compile time.

20. Testing React (More)

Q324. Unit vs integration vs E2E tests in frontend.

Unit: functions/hooks in isolation. Integration: a component with children, routing, mocked network. E2E: real browser against running app (Playwright/Cypress). Follow the testing trophy: most value in integration tests.

Q325. getBy vs queryBy vs findBy.

`getBy` throws if not found (assert present). `queryBy` returns null (assert absence). `findBy` is async, waits for appearance.

Q326. Which query priority does Testing Library recommend?

`getByRole` (with name) > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByDisplayValue` > `getByAltText` > `getByTitle` > `getByTestId`.

Q327. fireEvent vs userEvent.

userEvent simulates real interactions (focus, keydown, input, keyup, click sequence); fireEvent dispatches a single DOM event. Prefer userEvent.

Q328. How do you test a custom hook?

`const { result } = renderHook(() => useCounter()); act(() => result.current.inc()); expect(result.current.count).toBe(1);`

Q329. What is `act()` and why the warning?

Ensures all updates and effects are flushed before assertions. The warning appears when state updates happen outside act (often async updates you didn't await). Use `findBy`/`waitFor`.

Q330. How do you mock API calls?

MSW (Mock Service Worker) intercepts requests at the network level so the same mocks work in tests and the browser. Or `vi.mock`/`jest.mock` the API module.

```
server.use(http.get("/api/txns", () => HttpResponse.json([{ id: 1, rrn: "123" }])));
```

Q331. How do you test components using context, router or Redux?

Create a custom `renderWithProviders` that wraps with MemoryRouter, QueryClientProvider and store, accepting initial state.

Q332. How do you test timers (debounce)?

`vi.useFakeTimers(); ... vi.advanceTimersByTime(500);` and with userEvent use `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })`.

Q333. Snapshot testing: pros and cons.

Catches unexpected markup changes, but large snapshots get approved blindly. Use small, targeted snapshots or prefer explicit assertions.

Q334. How do you measure code coverage and what's a good target?

`vitest --coverage` / `jest --coverage`. Aim for meaningful coverage of critical logic (~70-80%), not 100% for its own sake.

Q335. What is visual regression testing?

Comparing screenshots between builds (Chromatic, Playwright screenshots, Percy) to catch unintended UI changes.

Q336. How do you test accessibility?

jest-axe / @axe-core/playwright, querying by role in tests, and manual keyboard and screen reader checks.

Q337. What is component-driven development with Storybook?

Building and documenting components in isolation with stories for each state; supports interaction tests and visual tests.

21. Next.js and Server-Side React

Q338. What is Next.js and why use it?

A React framework with file-based routing, SSR/SSG/ISR, Server Components, API routes, image/font optimisation and edge middleware.

Q339. App Router vs Pages Router.

App Router (`app/`) uses Server Components by default, nested layouts, streaming, server actions. Pages Router (`pages/`) uses `getServerSideProps/getStaticProps`.

Q340. What is a Server Component vs Client Component?

Server Components run only on the server, can be async and access DB/secrets, and ship no JS. Client Components (`"use client"`) handle interactivity, state, effects and browser APIs.

Q341. Can a Server Component import a Client Component and vice versa?

Server can import Client. Client cannot import Server, but can receive a Server Component as `children` props.

Q342. What are Server Actions?

Async functions marked `"use server"` callable from forms or client code; run on the server (mutations, revalidation). Must validate input and authorise like any API.

Q343. How does data fetching and caching work in the App Router?

`fetch` in Server Components with caching options (`cache`, `next: { revalidate: 60, tags: ["txns"] }`); `revalidatePath`/`revalidateTag` after mutations. Defaults changed across versions (Next 15 no longer caches fetch by default).

Q344. What is streaming SSR?

Server sends HTML in chunks; Suspense boundaries show fallbacks first and stream in content when ready (`loading.tsx`).

Q345. What is middleware in Next.js?

Runs before a request completes (edge runtime): auth redirects, rewrites, A/B testing, geo routing.

Q346. How do you handle environment variables in Next.js?

Server-only by default; prefix `NEXT_PUBLIC_` to expose to the browser (embedded at build).

Q347. What is ISR?

Incremental Static Regeneration: static pages regenerated in the background after `revalidate` seconds or on demand.

Q348. What are layout.tsx, page.tsx, loading.tsx, error.tsx?

Layout: persistent shared UI. Page: route content. Loading: Suspense fallback. Error: error boundary for the segment (client component).

Q349. How do dynamic routes work?

`app/txn/[id]/page.tsx` receives `params.id`; `generateStaticParams` pre-renders known IDs.

Q350. How do you do SEO in React apps?

SSR/SSG for crawlable HTML, `<title>`/meta tags (React 19 supports them in components, Next.js `metadata` export), semantic HTML, sitemap, canonical URLs, fast performance.

Q351. Remix / React Router 7 framework mode - how is it different?

Focuses on web fundamentals: loaders/actions per route, forms that work without JS, nested routing; now merged into React Router v7.

22. Security, Accessibility and Browser Questions

Q352. What security headers should a React app's server send?

Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options: nosniff, X-Frame-Options/frame-ancestors (clickjacking), Referrer-Policy, Permissions-Policy.

Q353. How do you avoid leaking secrets in a frontend?

Anything in the bundle is public. Keep API keys on the server, use a backend proxy, and scope public keys with domain restrictions.

Q354. What is clickjacking and how to prevent it?

An attacker loads your app in a hidden iframe and tricks clicks. Prevent with `frame-ancestors 'none'` CSP or X-Frame-Options DENY.

Q355. How do you secure a React app that handles card data?

Never handle raw PAN if possible: use payment provider hosted fields/iframes (reduces PCI scope), mask displayed PANs, no logging of sensitive data, HTTPS only, short session timeouts and auto-logout on inactivity.

Q356. How do you implement auto logout on inactivity?

Track user activity events (debounced) and reset a timer; on timeout, call logout and clear tokens; sync across tabs with BroadcastChannel or storage events.

Q357. How do you sync login state across tabs?

`BroadcastChannel("auth")` or listening to `storage` events when a logout flag changes.

Q358. What is ARIA and when should you use it?

Accessible Rich Internet Applications attributes add semantics for assistive tech. First rule of ARIA: use native HTML elements before ARIA roles.

Q359. How do you make a custom dropdown accessible?

Use `role="listbox"`/`option` or the combobox pattern, `aria-expanded`, `aria-activedescendant`, keyboard support (arrows, Enter, Esc, typeahead), focus management. Or use a headless library (Radix, React Aria).

Q360. What is focus trapping?

Keeping keyboard focus inside a modal while open (Tab cycles within) and restoring focus to the trigger on close.

Q361. How do you announce dynamic content to screen readers?

An `aria-live="polite"` region updated with status text ("5 results loaded").

Q362. What is the difference between `disabled` and `aria-disabled`?

`disabled` removes the element from tab order and blocks events. `aria-disabled="true"` keeps focusability (so screen reader users can discover it) but you must block actions manually.

Q363. Explain the critical rendering path.

HTML parsing to DOM, CSS to CSSOM, render tree, layout, paint, composite. JS can block parsing; use `defer`/`type="module"` and minimise render-blocking CSS.

Q364. What is a Service Worker and PWA?

A background script that intercepts network requests for caching/offline, push notifications. PWA = manifest + service worker + HTTPS for installable apps.

Q365. What is the same-origin policy?

Browsers restrict scripts from reading responses from other origins (scheme + host + port) unless CORS allows it.

Q366. What are micro-frontends?

Splitting a large frontend into independently deployed apps (Module Federation, single-spa). Pros: team autonomy. Cons: shared dependency versions, consistent UX, performance overhead.

23. More Output-Based React Puzzles

Q367. How many times does "render" log on one click?

```
function App() { const [a, setA] = useState(0); const [b, setB] = useState(0);
  console.log("render");
  return <button onClick={() => { setA(1); setB(1); }}>x</button>; }
```

Once per click (batched). On a second click the values are already 1, so React bails out (may log once more then stop).

Q368. What is displayed after clicking?

```
const [items, setItems] = useState([1, 2]);
const add = () => { items.push(3); setItems([...items]); };
```

`[1,2,3]` displays, but the original array was mutated: it works by accident and breaks memoised children or previous-state comparisons. Use `setItems(p => [...p, 3])`.

Q369. What does the Child log when Parent's count changes?

```
const Child = React.memo(({ onClick }) => { console.log("child"); return <button onClick={onClick}/>; });
function Parent() { const [c, setC] = useState(0); return <><Child onClick={() => {}} /><p onClick={() =>
    setC(c + 1)}>{c}</p></>; }
```

"child" logs every time: the inline arrow is a new reference, defeating memo. Wrap with useCallback.

Q370. What is the value of `count` shown after 3 seconds if you click 3 times quickly?

```
function C() { const [count, setCount] = useState(0);
  const click = () => setTimeout(() => setCount(count + 1), 3000);
  return <button onClick={click}>{count}</button>; }
```

1. All three timeouts captured `count = 0`. Fix: `setCount(c => c + 1)` gives 3.

Q371. What happens with this effect?

```
useEffect(() => { document.title = `Clicked ${count}`; });
```

Runs after every render (no deps). Fine here but wasteful; add `[count]`.

Q372. What is rendered: `{[1, 2, 3]}`, `{true}`, `{null}`, `{undefined}`, `{{a: 1}}`?

`123`, nothing, nothing, nothing, error ("Objects are not valid as a React child").

Q373. Why does this list re-mount all rows on every render?

```
{rows.map(r => <Row key={Math.random()} row={r} />)}
```

Random keys change each render, forcing unmount/remount (losing state, terrible performance). Use a stable id.

Q374. What is wrong here?

```
const [user, setUser] = useState({ name: "A", age: 1 });
setUser({ age: 2 });
```

`useState` setter replaces, it doesn't merge (unlike class setState). `name` is lost. Use `setUser(u => ({ ...u, age: 2 }))`.

Q375. Output order: parent and child effects?

Child effects run before parent effects (bottom-up), and cleanup runs in a similar order. Render goes top-down; effects go bottom-up.

Q376. Will this cause an error?

```
function List({ items }) { if (!items.length) return null; const [sel, setSel] = useState(null); ... }
```

Yes, the hook is called conditionally (after an early return). When items go from empty to non-empty, hook count changes. Move hooks above the return.

Q377. What happens if you set state during render?

```
function Bad() { const [x, setX] = useState(0); setX(1); return x; }
```

Infinite loop / "Too many re-renders". Setting state during render is only allowed conditionally for derived state adjustments (rare pattern).

Q378. What does `useState(Date.now())` vs `useState(() => Date.now())` differ in?

Both store the first value, but the first form calls `Date.now()` on every render (ignored after first). Lazy initialiser runs once. Matters for expensive initialisers.

Q379. What prints?

```
const ref = useRef(0);
const click = () => { ref.current++; console.log(ref.current); };
return <button onClick={click}>{ref.current}</button>;
```

Console increments 1, 2, 3, but the button text stays 0 because changing a ref doesn't re-render.

Q380. Two components using the same custom hook - do they share state?

No. Each call creates independent state.

Q381. What happens when context value is a new object every render and a consumer is wrapped in React.memo?

The consumer still re-renders (context bypasses memo).

Q382. What does this log?

```
useEffect(() => { console.log("A"); }, []);
useLayoutEffect(() => { console.log("B"); }, []);
console.log("C");
```

`C B A`. Render first, then layout effects (before paint), then passive effects.

Q383. Is this fetch effect correct?

```
useEffect(async () => { const d = await getData(); setData(d); }, []);
```

No, async effect returns a Promise; React warns. Define an inner async function and call it; handle cancellation.

Q384. Why does the input lose focus on every keystroke?

Usually a component defined inside another component, or a changing `key` on the input/its parent, causing remount on each render.

Q385. What happens if a key changes on a component with an effect?

The old instance unmounts (cleanup runs), a new one mounts (effect runs again with fresh state).

24. More Machine-Coding Problems

Q386. Build a star rating component.

```
function Stars({ value, onChange, max = 5 }) {
  const [hover, setHover] = useState(0);
  return <div role="radiogroup">{Array.from({ length: max }, (_, i) => i + 1).map(n => (
    <button key={n} role="radio" aria-checked={value === n} onMouseEnter={() => setHover(n)}
    onMouseLeave={() => setHover(0)}
      onClick={() => onChange(n)}>{n <= (hover || value) ? "\u2605" : "\u2606"}</button>))}</div>;
}
```

Q387. Build an accordion (one panel open at a time).

Store `openId` in state; clicking a header sets `openId` to its id or null if already open. Use `aria-expanded` and `aria-controls`.

Q388. Build tabs with keyboard support.

State `active` index; `role="tablist"`, `role="tab"` with `aria-selected`, Arrow keys move focus and selection, panels `role="tabpanel"`.

Q389. Build a progress bar that animates to a value.

A div with width `${pct}%` and CSS transition; clamp 0-100; `role="progressbar"` with aria-valuenow.

Q390. Build a typeahead with keyboard highlighting.

State: query, results, activeIndex. ArrowDown/Up change activeIndex modulo length; Enter selects; Escape closes; `aria-activedescendant` on the input.

Q391. Build a traffic light that cycles red -> green -> yellow with durations.

```
const cfg = { red: [4000, "green"], green: [3000, "yellow"], yellow: [1000, "red"] };
function Light() { const [c, setC] = useState("red");
  useEffect(() => { const t = setTimeout(() => setC(cfg[c][1]), cfg[c][0]); return () => clearTimeout(t);
    }, [c]);
  return <div className={`light ${c}`} />; }
```

Q392. Build a pagination component with ellipses.

Compute page numbers: always show first, last, current +/- 1, and insert "..." for gaps. Disable prev/next at bounds.

Q393. Build a toast notification system.

Context provider holds a list of toasts; `useToast()` exposes `show(msg, type)`; each toast auto-removes after timeout; rendered in a portal; `aria-live`.

Q394. Build a drag-and-drop sortable list.

Native HTML5 drag events (`draggable`, `onDragStart` store index, `onDragOver` preventDefault, `onDrop` reorder array) or dnd-kit for accessibility and touch.

Q395. Build an image carousel.

Index state, prev/next with wrap-around, optional autoplay interval paused on hover, dots, lazy-loaded images, swipe support.

Q396. Build a nested comments component with replies.

Recursive `Comment` rendering its replies; adding a reply updates the tree immutably (find by id recursively) or use a normalised map `{ id: { text, childIds } }`.

Q397. Build a shopping cart with quantity and totals.

useReducer with add/remove/updateQty; derived total via useMemo; persist to localStorage; format currency with Intl.

Q398. Build a data grid with column filters and sort, client-side.

Store filters and sort in state; derive visible rows with useMemo: filter, then sort with a comparator; virtualise if large.

Q399. Build a "select all" checkbox list with indeterminate state.

Selected ids in a Set; header checkbox `checked` when all selected; set `ref.current.indeterminate = some && !all` in an effect.

Q400. Build a countdown to a target date.

Compute remaining from `target - Date.now()` every second; format d/h/m/s; clear at zero.

Q401. Build a file uploader with drag-drop and progress.

Drop zone handling `onDragOver`/`onDrop`, list files with validation, upload each with axios `onUploadProgress`, cancel per file with AbortController.

Q402. Build a chat UI that auto-scrolls to the latest message.

Ref on the last message; in `useLayoutEffect` after messages change, `scrollIntoView` only if the user was already near the bottom.

Q403. Build a light/dark theme toggle with context.

ThemeContext providing `theme` and `toggle`; set `document.documentElement.dataset.theme`; persist preference.

Q404. Build a password strength meter.

Score by length and character classes; show bar colour and text; validate with debounced input.

Q405. Build a Kanban board.

Columns from a map `{ status: ids[] }`, cards by id; drag between columns updates both arrays; persist with optimistic update to API.

Q406. Build a form with conditional fields (e.g. show GST if business).

Watch a field value (RHF `watch("type")`) and conditionally render and validate dependent fields with Zod `discriminatedUnion` or `refine`.

Q407. Build a calculator.

State for display, previous value and operator; handle digits, decimal, operators, equals, clear; avoid `eval`.

Q408. Build a tic-tac-toe game.

Board array of 9; current player derived from count of moves; winner computed from 8 winning lines; history for time travel (classic React tutorial).

Q409. Build a memory card game.

Shuffle pairs, track flipped indices (max 2), matched set; when two flipped, compare and flip back after delay if not matching.

Q410. Build an Excel-like editable table cell.

Display mode vs edit mode per cell (state `editing: {row, col}`); Enter commits, Escape cancels; keyboard navigation with arrow keys.

25. Real-World Scenarios (More)

Q411. The product owner says the app is "slow on first load". What do you check?

Bundle size and code splitting, render-blocking resources, API waterfalls (sequential calls), large images, no caching headers/CDN, SSR possibility, Lighthouse report.

Q412. A page makes 8 API calls on load. How do you improve it?

Run in parallel, cache with TanStack Query, combine into a BFF/aggregate endpoint, defer non-critical widgets, prefetch on navigation.

Q413. How do you handle API versioning changes on the frontend?

Centralised API client with typed responses, adapter functions mapping API DTOs to UI models so changes are isolated, feature flags for gradual rollout.

Q414. How do you handle feature flags?

A flag service (LaunchDarkly, Unleash or config endpoint) loaded at startup, `useFlag("newRecon")` hook, remove flags after rollout.

Q415. How do you structure environment-specific config for dev/UAT/prod?

Build-time env vars per environment, or runtime config by fetching `/config.json` at startup so one build can deploy to all environments.

Q416. Users report a blank white screen in production. How do you debug?

Check console errors via logging tool (Sentry), source maps, Error Boundary coverage, failed chunk loads after deployment (old cached index.html requesting deleted chunks: handle with retry/reload), browser compatibility.

Q417. After deployment, users get "ChunkLoadError". Why and fix?

Users with the old app request old hashed chunks that were removed. Keep previous assets for a while, set `index.html` to no-cache, and catch lazy import failures to reload the page once.

Q418. How do you implement audit-friendly approval workflow UI (maker-checker)?

Maker creates request, checker sees pending queue; buttons hidden/disabled based on role and status; confirmation dialogs with remarks; show history timeline from API; optimistic locking to prevent double approval.

Q419. How do you show large reconciliation reports with drill-down?

Summary dashboard (counts, amounts by status) from aggregated API, click to filter detail grid (server-side paging), export via backend-generated Excel for large data, URL keeps the drill-down state.

Q420. How would you export data to Excel/CSV from React?

Small data: SheetJS or generate CSV client-side. Large data: trigger backend export job, poll status, download the file.

Q421. How do you handle a form that must save drafts automatically?

Debounced autosave (e.g. 2s after last change) to API or localStorage, show "Saving.../Saved" status, handle conflicts with version numbers.

Q422. How do you handle concurrent edits by two users?

Send a version/ETag with updates; server returns 409 on conflict; UI shows a merge dialog or reloads latest.

Q423. How would you build a dashboard with 12 widgets that load independently?

Each widget has its own query and error boundary and Suspense/skeleton, so one failing or slow widget doesn't block others; lazy-load below-the-fold widgets.

Q424. How do you migrate a large class-component codebase to hooks?

Incrementally: new code in hooks, convert leaf components first, extract shared logic into custom hooks, add tests before refactoring, keep error boundaries as classes.

Q425. How do you upgrade React 17 to 18/19 safely?

Switch to `createRoot`, fix StrictMode double-effect issues, check libraries for compatibility, remove deprecated APIs (string refs, legacy context, defaultProps on functions for 19), run tests and smoke test.

Q426. How do you migrate from CRA to Vite?

Add Vite config and index.html at root, rename env vars to `VITE_`, replace `process.env` with `import.meta.env`, update Jest to Vitest optionally, fix absolute imports and SVG imports.

Q427. How do you handle errors from a third-party script (analytics) breaking your app?

Load asynchronously, wrap calls in try/catch, isolate in iframe if needed, and use CSP to control sources.

Q428. How would you design a reusable table component used across 20 screens?

Headless approach (TanStack Table) for logic: sorting, filtering, pagination, column visibility, row selection; your own styled rendering layer; server-side mode support; column definitions passed as config.

Q429. How do you handle a slow API search where users type fast?

Debounce input, cancel previous requests, show loading indicator, cache by query, minimum length, and server-side indexing.

Q430. How do you ensure consistent code quality in a React team?

ESLint (react-hooks plugin), Prettier, TypeScript strict, Husky + lint-staged pre-commit, PR reviews, CI running tests and build, Storybook for shared components.

Q431. How do you deploy a React app?

`npm run build` produces static files; host on IIS/Nginx/S3+CloudFront/Azure Static Web Apps; configure SPA fallback, caching (hashed assets long cache, index.html no-cache), gzip/brotli, HTTPS, CI/CD pipeline.

Q432. How do you host React and a .NET API together?

Serve React build from `wwwroot` with `app.UseStaticFiles()` and `app.MapFallbackToFile("index.html")`, or separately with CORS / reverse proxy under the same domain (`/api` -> backend).

26. Experience and Project Discussion Questions

Q433. Walk me through the architecture of your current React project.

Structure the answer: purpose of the app, tech stack (React, TypeScript, state/data libraries, UI kit), folder structure, API layer and auth, key features you built, performance or scale challenges and how you solved them, testing and deployment.

Q434. Tell me about a difficult bug you fixed in React.

Use STAR: Situation (e.g. dashboard showed wrong totals after filter changes), Task, Action (profiled, found stale closure / race condition, added AbortController and functional updates), Result (correct data, faster renders).

Q435. How did you improve performance in a project?

Give measurable results: "Reduced initial bundle from 3.2MB to 900KB with route-based splitting and replacing moment.js; grid render time from 2s to 150ms using virtualisation."

Q436. How do you decide between building a component vs using a library?

Consider complexity (date pickers, grids are hard), accessibility, bundle size, customisation needs, maintenance, licence.

Q437. How do you review a React pull request?

Check correctness and edge cases, hooks rules and dependencies, unnecessary state/effects, accessibility, naming, tests, performance pitfalls, security (dangerouslySetInnerHTML, secrets).

Q438. What React features or trends are you following?

React 19 (Actions, use, useOptimistic), React Compiler, Server Components, React Router 7 framework mode, TanStack ecosystem, signals debate, Vite/Rolldown tooling.

27. Rapid-Fire Round (One-Liners)

Q439. What is a controlled component in one line?

An input whose value is driven by React state via `value` + `onChange`.

Q440. What is reconciliation in one line?

React's process of diffing the new element tree against the previous one to compute minimal DOM updates.

Q441. What does the empty dependency array mean?

Run the effect once after the initial mount (cleanup on unmount).

Q442. Can you use hooks in class components?

No. Wrap the class with a function component or HOC that uses the hook and passes values as props.

Q443. What is the default behaviour of an `<a>` in React Router?

Full page reload; use `<Link>` or `<NavLink>` for client-side navigation.

Q444. What does NavLink add over Link?

Active state styling (`className={({ isActive }) => ...}`).

Q445. What is `React.StrictMode` double rendering in production?

It doesn't happen in production.

Q446. What is the virtual DOM made of?

Plain JavaScript objects (React elements) and Fiber nodes.

Q447. How do you force a component to re-render?

Update its state (`const [, force] = useReducer(x => x + 1, 0)`), or change its key to remount. Usually a design smell.

Q448. Which hook replaces componentDidCatch?

None; error boundaries still need a class (or react-error-boundary).

Q449. What is a pure component?

One that renders the same output for the same props and state; `React.memo`/PureComponent skip re-render when shallowly equal.

Q450. What does `React.Children.only` do?

Verifies children has exactly one element and returns it, otherwise throws.

Q451. What is the maximum number of root nodes a component can return?

One root (wrap in a Fragment for multiple).

Q452. Can props be functions?

Yes, callbacks and render props are functions passed as props.

Q453. What is lifting state in one line?

Moving shared state to the closest common ancestor.

Q454. What is a side effect in React?

Any interaction with the outside world: data fetching, subscriptions, timers, DOM manipulation, logging.

Q455. Where do you put API base URLs?

Environment variables / runtime config, used by a single API client.

Q456. What is hot module replacement?

Updating modules in the running app during development without a full reload, preserving state (React Fast Refresh).

Q457. What is Babel/SWC's role in React?

Transpiles JSX and modern JS/TS to browser-compatible JavaScript.

Q458. What is the difference between npm, yarn and pnpm?

Package managers; pnpm uses a content-addressable store with symlinks (fast, disk-efficient), yarn and npm have lockfiles and workspaces.

Q459. What is package-lock.json for?

Locks exact dependency versions for reproducible installs; commit it.

Q460. What are peer dependencies?

Dependencies a library expects the host app to provide (e.g. `react`) so only one copy is installed.

Q461. What is semantic versioning?

MAJOR.MINOR.PATCH: breaking, features, fixes. `^1.2.3` allows minor/patch updates; `~1.2.3` allows patch only.

Q462. What is a monorepo and tools for it?

Multiple packages/apps in one repo sharing code; tools: Nx, Turborepo, pnpm workspaces.

Q463. What does `npm run build` produce in Vite?

Optimised static assets in `dist/` with hashed filenames, minified JS/CSS, code-split chunks.

Q464. What is source map?

A file mapping minified code back to original source for debugging; upload to error tracking, don't expose publicly if sensitive.

Q465. How do you add absolute imports in Vite?

`resolve.alias` in vite.config and `paths` in tsconfig (e.g. `@/` -> `src/`).

Q466. What is ESLint plugin react-hooks for?

Enforces rules of hooks and exhaustive dependencies.

Q467. What is the difference between `useEffect` and an event handler for side effects?

Event handlers run in response to specific user actions; effects run because rendering happened and something must be synced.

Q468. What's the role of `key` when rendering a single component?

It resets the component when the key changes (state reset pattern).

Q469. What is a portal's event bubbling behaviour?

Events bubble according to the React tree, not the DOM tree.

Q470. What is the React element `$$typeof`?

A Symbol that marks objects as genuine React elements, preventing JSON injection from being treated as elements (XSS protection).

Q471. What is "lifting content up" (composition) for performance?

Passing JSX as props/children from a higher component so it isn't recreated when a lower component's state changes.

Q472. What is an uncontrolled component's default value?

Set with `defaultValue` / `defaultChecked`, not `value`.

Q473. What is `React.Suspense` without lazy or data used for?

Nothing by itself; it only shows fallbacks when a child suspends.

Q474. What does `startTransition` do outside a component?

The standalone `startTransition` marks updates as transitions where you can't use the hook (e.g. in a store or router).

Q475. What is `React.version`?

The running React version string, useful for debugging duplicate React copies.

Q476. What causes "Invalid hook call" error?

Calling hooks outside components, mismatched react/react-dom versions, or two copies of React in the bundle (common in linked packages/monorepos).

28. Bonus Tricky Questions

Q477. Why can't you call `setState` inside the render of another component?

React warns "Cannot update a component while rendering a different component" because it breaks render purity. Move the update to an effect or event handler.

Q478. Why does `useEffect` with a function dependency run every render?

Functions defined in the component body are recreated each render. Move it inside the effect, outside the component, or wrap with useCallback.

Q479. How do you compare old and new props in a function component?

Store previous props in a ref (`usePrevious`) or compare in an effect with the dependency that changed.

Q480. What happens if you return a cleanup that isn't a function from useEffect?

React warns; only a function (or undefined) is valid. Common with async effects returning promises.

Q481. Does React re-render when you set state to the same object after mutating it?

No. Same reference means bailout, so the mutation never shows. Always create new objects.

Q482. Why does `useState` initial value from props not update?

The initial value is used only on mount. Use a key to reset or derive during render.

Q483. Can a component be both controlled and uncontrolled?

No. Choose one; switching triggers a warning. Libraries offer "controllable" props: controlled if `value` provided, else internal state.

Q484. Why do two consecutive `navigate()` calls only keep the last?

Navigation updates are batched like state; the last wins. Compute the final destination first.

Q485. What does `key` on a Context Provider do?

Same as any element: changing it remounts the provider and its subtree, resetting all state below.

Q486. Why might `onChange` on a custom input not fire with `value` set programmatically?

Programmatic `input.value = x` doesn't fire React's onChange. Update state instead, or dispatch a native input event via the value setter (hack used in tests).

Q487. What is the difference between `React.lazy` and dynamic import alone?

`import()` returns a promise for a module; `React.lazy` wraps it so React can suspend rendering until it loads (default export required).

Q488. Why is `React.lazy` with a named export failing?

Lazy expects a default export. Use `lazy(() => import("./X").then(m => ({ default: m.Named })))`.

Q489. How do you preload a lazy component?

Call the same `import()` early (on hover) so the module is cached when lazy renders.

Q490. How do you handle a Suspense waterfall?

Sibling components inside one Suspense boundary load in parallel; nested fetching parents-then-children cause waterfalls. Start fetches early (loaders, prefetch) and render as you fetch.

Q491. What happens if a Suspense boundary is missing above a lazy component?

React throws an error: a component suspended with no Suspense fallback above it (in React 18+ the root may show nothing and warn).

Q492. What is `ErrorBoundary` reset strategy?

Reset by key change (e.g. route change) or `resetErrorBoundary` with `resetKeys` in react-error-boundary after the user retries.

Q493. Why are styles from CSS Modules not applied?

File must be named `*.module.css` and class used as `styles.className`; hyphenated names need `styles["my-class"]`.

Q494. Why does `useEffect` not run on server rendering?

Effects run only in the browser after hydration. Server renders HTML without running effects (useLayoutEffect warns on server in older React).

Q495. How do you detect if the code runs on the client in SSR?

`typeof window !== "undefined"` or set `isClient` in a `useEffect`. Avoid branching rendering on it to prevent hydration mismatch.

Q496. What is "prop spreading" and its risks?

`<Comp {...props} />` passes everything, including unknown attributes to the DOM (warnings) and accidental overrides. Destructure what you need and spread the rest intentionally.

Q497. What is the difference between `useRef()` and `createRef()`?

`createRef` creates a new ref object every call (used in classes in the constructor); `useRef` returns the same object across renders.

Q498. How do you focus an input on mount?

`autoFocus` attribute, or `useEffect(() => inputRef.current?.focus(), [])`.

Q499. How do you measure the height of dynamic content?

Callback ref + `ResizeObserver` to react to size changes, updating state in a layout effect.

Q500. Why does my modal close when I click inside it?

The click bubbles to the backdrop handler. Stop propagation on the content or check `e.target === e.currentTarget` in the backdrop handler.

Q501. How do you prevent double API calls in development?

They come from StrictMode's double effect run. Use a data library that dedupes, AbortController cleanup, or accept it in dev; it doesn't happen in production.

Q502. Why are environment variables undefined in Vite?

They must start with `VITE_` and be accessed via `import.meta.env`; restart the dev server after changes.

Q503. How do you avoid prop name collisions with HTML attributes?

Avoid naming custom props like native attributes (`size`, `type`, `color`) when spreading to DOM, or filter them out.

Q504. Why does a component re-render when its parent passes the same primitive props?

Because the parent re-rendered; props equality doesn't stop re-render without React.memo.

Q505. What is "key" warning inside Fragments in a map?

Each fragment in a list needs a key: use `<React.Fragment key={id}>`, since `<>` can't take keys.

Q506. How do you share a function between sibling components?

Define it in the parent and pass it down as a prop to both, or put it in context/custom hook.

Q507. Why does `this.state` update not reflect immediately in class components?

`setState` is async and batched; use the callback `setState(updater, () => console.log(this.state))` or `componentDidUpdate`.

Q508. How do you get the latest state in an event handler after awaiting?

The handler's closure holds the old render's state. Use a ref mirroring state, or functional updates for computations.

Q509. What is the difference between `onBlur` and `onFocusOut` in React?

React's `onBlur`/`onFocus` actually use focusout/focusin (they bubble in React), so you can handle them on a parent.

Q510. How do you implement a "controlled" open state for a dropdown that also works uncontrolled?

`const isControlled = open !== undefined; const [inner, setInner] = useState(defaultOpen); const actual = isControlled ? open : inner;` and call both `onOpenChange` and `setInner`.

Q511. Why does SVG imported as a component fail in Vite?

Vite imports SVG as a URL by default; use `vite-plugin-svgr` and `import Icon from "./a.svg?react"`.